Agentic RL on Terminal-Bench with slime and AgentCore Runtime

Hands-on guidance, grounded in a working Terminal-Bench RL demo

Kaige Yang — Senior Applied Scientist, AWS

September 30, 2026

Contents

1 Introduction **2**

2 Agentic RL and its reference architecture **3**
2.1 Where agentic RL is used . . . 5
2.2 The harness — the agent loop that owns the episode . . . 6
2.3 The proxy — every model call is the policy under training . . . 8
2.4 Token-in-token-out (TITO) — the proxy's core responsibility . . . 10
2.5 When one episode becomes many samples — subagents, summarization, and advantage assignment . . . 13
2.6 slime — the trainer, and its asynchronous rollout . . . 16
2.7 Scaling the loop — inference, harness, and sandbox in balance . . . 18

3 Tasks frontier labs run in a sandbox **25**
3.1 What the frontier reports describe . . . 26
3.2 Task families in detail . . . 27
3.2.1 Single-turn, generation-bound families . . . 28
3.2.2 Iterative, few-turn families . . . 29
3.2.3 Multi-turn, observation-heavy families . . . 29
3.2.4 Long-horizon, session-bound families . . . 30
3.3 Language and environment per family — and how one microVM serves them all . . . 31
3.4 The one family that stresses the image: SWE and the microVM's arm64 + 2 GB limits . . . 35
3.5 One image per Runtime, but a different env per row: how AgentCore reconciles it . . . 37

4 Foundations of AgentCore Runtime **38**
4.1 The microVM at a glance — what it is, what runs in it, what it supports, how you customize it . . . 38
4.2 Runtime, session, and microVM — three concepts, precisely . . . 39
4.2.1 A mixed-family batch — how many Runtimes, sessions, and microVMs, and who picks the image? . . . 41
4.3 microVM vs a Docker container vs E2B . . . 42
4.4 Two compute types — managed serverless (microVM) vs Instances (GPU EC2) . . . 43

4.5 How AgentCore manages the lifecycle — Runtime, session, microVM, and GPU instance, end to end . . . 46
4.6 Building a customized Runtime — bring your own container, end to end 50

5 Using AgentCore as the sandbox in an agentic RL workflow 53
5.1 Where AgentCore attaches to slime — hooks, not a fork . . . 53
5.2 The call path — one diagram . . . 54
5.3 Key code — three pieces . . . 54
5.4 Who controls the AgentCore lifecycle — deploy driver vs. the hook . . . 56
5.5 Ending the session at episode end — explicit teardown, and what "end" means per substrate . . . 58
5.6 One harness, many task families . . . 60
5.7 Reading the episode shape into a config . . . 61
5.8 The episode, end to end . . . 61
5.9 Scaling, sharing, and network isolation . . . 62
5.10 Can AgentCore host every frontier sandbox? . . . 63
5.11 Related work — DeepSeek DSec, a purpose-built sandbox platform . . . 63

6 The Terminal-Bench demo 67
6.1 Which sandbox does a task need? Three tiers . . . 67
6.2 Use case covered . . . 68
6.3 Why this demo . . . 69
6.4 Design considerations . . . 69
6.5 The pool . . . 70
6.6 Hiding the container launch: pre-open and heartbeat . . . 71
6.7 Scaling the harness with inference and the sandbox: the harness tier . . . 73
6.8 Run it step by step . . . 74

7 The Terminal-Bench demo in depth — pipeline, metrics, and measured results 75
7.1 The pipeline — one episode, one step . . . 76
7.2 Sizing the config — response length per turn, and turns per episode . . . 79
7.3 Measured — the 2026-09-23 smoke (first folder) . . . 79
7.4 Measured — the 2026-09-30 smoke (current folder) . . . 80
7.5 Measured — the 2026-09-30 re-run (current code) . . . 81
7.6 Measured — Qwen3.5-9B, one full epoch (2026-10-11) . . . 81
7.7 Experiment plan — training to convergence . . . 83

8 History: removed demos (2026-09-30) 84

9 Open problems in production agentic RL — infrastructure and algorithms 85
9.1 Infrastructure and systems 86
9.2 Algorithms 87
9.3 Takeaways 89

10 Summary 89

11 References — papers & models 90

12 References — AgentCore docs (1/2) 90

13 References — AgentCore docs (2/2) 90

14 References — AgentCore SDK, CLI & samples 90

15 References — production agentic RL (open problems) 91

1 Introduction

This report is a **hands-on guide for AWS customers doing agentic RL**, built around one working demo: **RL on Terminal-Bench with slime and AgentCore Runtime** (written for `code/demos/demo_cpu_terminalbench`, now `code/demos/demo_terminalbench_kernelbench`; see the update note above). The policy is Qwen3-4B, trained by slime with fully-async GRPO. The sandbox is an **image pool of 218 AgentCore arm64 microVM Runtimes in VPC mode, one per Terminal-Bench task**, with a fresh session per episode. The agent loop runs **inside that session**, and the reward is the task's own hidden pytest oracle. The claims are grounded in the **official AgentCore docs** [D1] and in this project's run logs.

After reading it, you will know:

- **The reference architecture of agentic RL**: how the trainer, inference engine, proxy, harness and sandbox fit together, and which seams AgentCore fills (§"Agentic RL and its reference architecture").
- **What tasks labs train in a sandbox, and how the reward works**: the verifiable task families, each scored by a programmatic **RLVR** checker rather than an LLM judge, and why Terminal-Bench is the hardest sandbox shape among them (§"Tasks frontier labs run in a sandbox").
- **What AgentCore Runtime *is***: the three concepts you must not conflate (**Runtime**, **session**, **microVM**), their **1 : N : 1** cardinality, the two compute types, the **~15-minute idle timer**, the **2 GB arm64 image cap**, and how to build your own Runtime (§"Foundations of AgentCore Runtime").
- **How AgentCore plugs into slime without a fork**: one generate hook, a sandbox client that turns each operation into one `InvokeAgentRuntime`, and the agent loop in the session, dialling the trainer's adapter back over the VPC (§"Using AgentCore as the sandbox in an agentic RL workflow").
- **How to run agentic RL on hundreds of per-task images** (§"The Terminal-Bench demo"):
  - one image and one Runtime per task, built from the task's own Dockerfile;
  - strict row → image → Runtime routing;
  - a fresh microVM per episode;
  - **look-ahead pre-open** to hide the cold start;
  - a **heartbeat** to survive the trainer's weight-sync stall;

  - a **harness tier** that scales the agent loop with the sandbox rather than inside the trainer.

- **What the runs measured and what is still open**: the 2026-09-23 and 2026-09-30 smokes, the convergence plan (§"The Terminal-Bench demo in depth"), and the production open problems (§"Open problems in production agentic RL").

Scope. AgentCore is the execution **sandbox**, **slime** is the RL trainer, and the **harness runs inside the sandbox session**. The harness is the mini_claude agent loop, one per episode, and it calls back to the trainer's per-engine adapter. It therefore scales with the sandbox, and the trainer keeps only a thin dispatch tier. Every other demo the project built, including a SWE-bench Verified pool, was removed on 2026-09-30; they are summarized once in §"History: removed demos (2026-09-30)".

Roadmap. The report proceeds in this order:

1. the **reference architecture** of agentic RL and how slime drives it (§2);
2. the **verifiable task families** and their rewards (§3);
3. the **foundations of AgentCore Runtime** (§4);
4. **wiring AgentCore into slime** as the sandbox (§5);
5. **the Terminal-Bench demo** (§6) and its **in-depth results** (§7);
6. the removed demos (§8);
7. the **open problems** (§9) and a **summary** (§10).

2 Agentic RL and its reference architecture

What is agentic RL? In *agentic* reinforcement learning the model under training is not asked for a single answer — it acts as an **agent** over many turns against an **executable environment**: it writes code, runs a command, reads the result, and tries again. Only when the trajectory ends does a **programmatic checker** turn the outcome into a scalar reward (tests pass, result-set matches, flag verified) — **no LLM judge** [1, 2]. This is reinforcement learning with *verifiable* rewards (RLVR): the environment, not a model, decides who was right.

Training that loop end-to-end uses a **disaggregated** design (following Agent Lightning [1]): a training side (trainer + inference engine) and an agent side (harness

+ sandbox), joined by an LLM-proxy / rollout-collection gateway. The reference architecture below traces the full cycle — a **generation path** across the top and a **training-data path** back along the bottom.

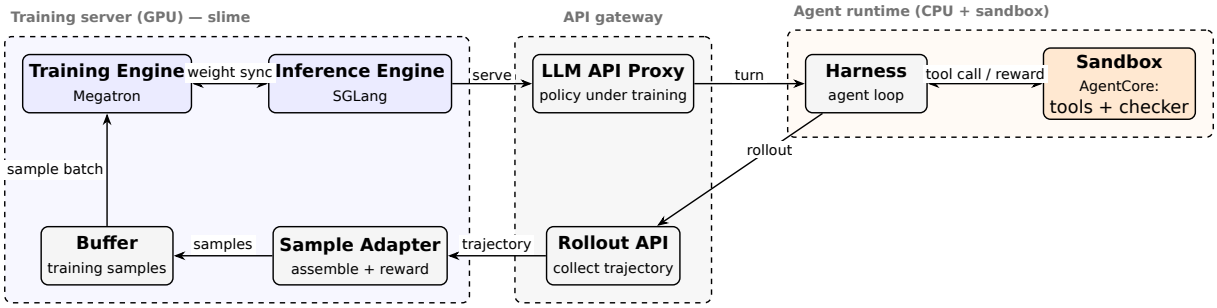


Figure 1: Reference architecture for (harnessed) agentic RL, following the disaggregated design of Agent Lightning [1] with slime [9] as the training server. Three domains: a **training server** (GPU), an **API gateway**, and an **agent runtime**. **Generation lane (top, $\rightarrow$):** the training engine syncs weights to the inference engine; the LLM API proxy fronts inference so every turn is served by the policy under training; the harness drives the agent loop; the sandbox (AgentCore) runs each tool call and the checker. **Training-data lane (bottom, $\leftarrow$):** the rollout is collected via the Rollout API, the Sample Adapter assembles it from the recorded token IDs (no re-tokenization — the TITO property) and attaches the reward to form training samples, which accumulate in the Buffer; the trainer draws a sample batch and updates — closing the loop.

The core components.

- **Trainer** — the learner (here **slime** driving **Megatron**, on GPU). It samples prompts, collects finished trajectories with their rewards, computes the policy-gradient update, and periodically pushes fresh weights to the inference engine.
- **Inference engine** — the generation server (here **SGLang**, on GPU) that produces the model's turns at rollout speed. It holds the *current* policy weights; the trainer refreshes them via **weight sync** so rollouts always reflect the latest policy.
- **Proxy** — a thin adapter (CPU) fronting the inference engine so that every model call the harness makes is answered by the **policy under training** (not some external API). It is the seam that lets an off-the-shelf agent loop drive an in-training model.
- **Harness** — the **agent loop** (CPU): it holds the conversation, calls the proxy for each turn, extracts the model's proposed action (a code block, a tool call),

submits it to the sandbox, and feeds the result back as the next observation until the episode ends.
- **Sandbox** — the **isolated execution environment** (CPU, or GPU for CUDA work) where the model's untrusted output actually runs (write → run → read) and where the **checker scores the reward**. It must be ad-hoc and isolated per episode.

The training-data path. The bottom of Figure 1 shows how a finished episode becomes a gradient step. The **Rollout API** collects the trajectory (the sequence of LLM request–response pairs the harness made); the **Sample Adapter** assembles those calls from their recorded token IDs (no re-tokenization — §"Token-in-token-out") and attaches the episode's reward and advantage, turning one rollout into one or more **training samples**; those land in the **Buffer**; and the trainer draws a **sample batch** from the buffer for each update. In *harnessed* agentic RL the training engine never sees the environment directly — only this stream of samples reconstructed from the proxy's request/response log [1].

2.1 Where agentic RL is used

The same loop — act in an environment, score with a checker — covers a wide range of tasks. The common thread is a **verifiable reward**: an environment (not a human or an LLM judge) decides success. Representative use cases:

- **Software engineering (SWE)** — edit a real repository to resolve an issue; reward = the project's test suite turns green (e.g. SWE-bench [6]).
- **Code generation & competitive programming** — write a function that passes hidden unit tests (e.g. MBPP [8]).
- **GPU-kernel / systems** — write a CUDA kernel and optimize it; reward = numerical `allclose` plus a speedup target (e.g. KernelBench [7]).
- **Math & logical reasoning** — produce an answer checked for exact/numeric equality against ground truth.
- **Tool use / function calling** — orchestrate multi-step API or tool calls; reward = correct final result *and* the right tools invoked.
- **SQL & data analysis** — generate a query whose result-set matches the expected rows.
- **RAG & search agents** — retrieve then answer; reward = answer grounded in the retrieved facts.
- **Web & computer use** — drive a browser or GUI to complete a task.

- **Security / CTF** — exploit a challenge and capture the flag; reward = the checker validates the flag.
- **Multi-agent & long-horizon workflows** — several agents or many turns cooperate toward a verifiable end state.

Every one of these needs an **isolated place to execute untrusted output and run the checker** — the sandbox seam this report is about.

2.2 The harness — the agent loop that owns the episode

What it is. The harness is the **agent loop**: the orchestration code that runs *one* episode end to end. In this project it is the tool-calling **mini_claude** loop (`write_file` / `bash` / `read_file` over the Anthropic Messages API, `mini_agent.py`), and it is **task-agnostic** — the same loop serves every task family; only the task's prompt, seed files, and checker change. It runs **inside the episode's own sandbox session**: `in_session_harness.py` writes a stdlib runner into the session, starts it and long-polls it, and every model turn calls back to the trainer's `AnthropicAdapter`, which records the exact tokens. Two reasons put it there rather than in the trainer. The harness has to scale **along with** inference and the sandbox, or a few coroutines silently become the throughput ceiling. And the unit that already scales per episode is the sandbox session, so one loop per session scales with it — DSec's "move the agent loop out of the trainer" [31].

How it works with the other components. The harness sits between the proxy and the sandbox and is the only component that sees a whole episode:

- **↔ Proxy:** for each turn it sends the running conversation to the proxy and receives the model's response (the policy under training).
- **↔ Sandbox:** it extracts the model's proposed action (a code block, a tool call) and executes it in the sandbox, then reads back stdout / exit-code as the next observation.
- **↔ Task / checker:** it seeds the workspace from the task and, at the end, runs the checker (in the sandbox) to get the reward.
- **→ Rollout / Sample Adapter:** the trajectory it produces (the proxy-logged calls + the final reward) becomes the training sample.

Step-by-step example — one MBPP episode from the harness's view (a simplified sketch of the loop's logic; the shipped loop is the tool-calling mini_claude, run inside the sandbox session).

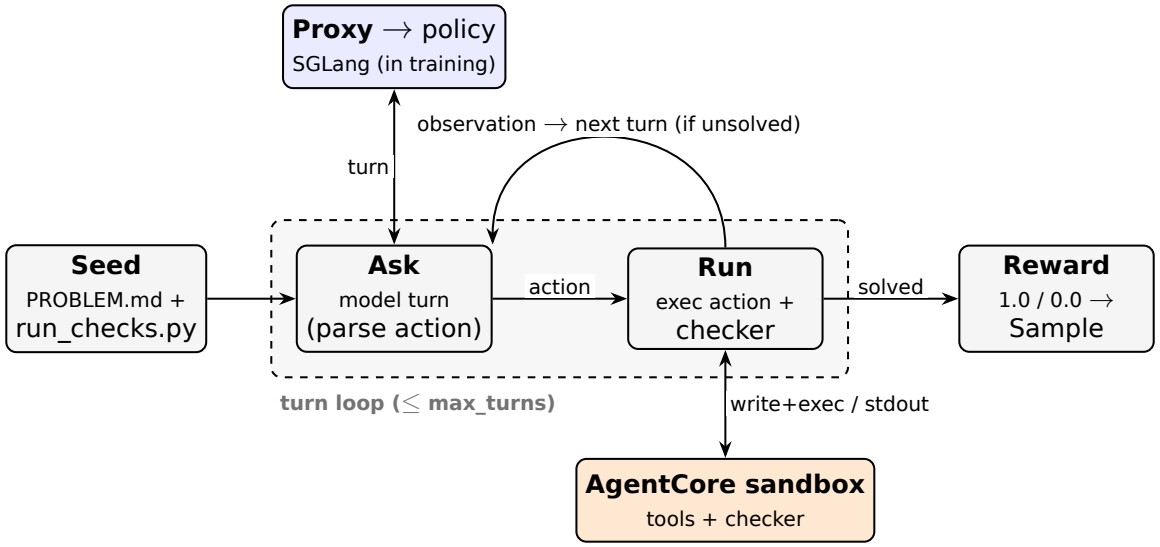


Figure 2: The harness's episode loop (`run_episode`). It **seeds** the sandbox once, then repeats a **turn loop**: ask the proxy for a model turn, parse the proposed action, run it (and the checker) in the AgentCore sandbox, and feed the observation back for the next turn. The loop exits when the checker passes or `max_turns` is reached; the final **reward** and the loss-masked trajectory become one training `Sample`. The proxy (policy under training) and the sandbox are the only two services the loop calls each turn.

```python
async def run_episode(task, policy, sandbox):          # one episode
    # 1. SEED: write the problem + the checker into the fresh sandbox
    for name, content in task.setup_files():           # e.g. PROBLEM.md
        await sandbox.write_file(name, content)
    await sandbox.write_file("run_checks.py", task.runner_source())

    messages = task.initial_messages()                 # system + PROBLEM.md
    solved = False
    for _ in range(task.max_turns):                    # 2. TURN LOOP
        text = await policy.complete(messages)         #    (a) ask proxy -> policy
↪  turn
        code = task.extract_submission(text)           #    (b) parse the action
↪  (fenced code)
        await sandbox.write_file("solution.py", code)  #    (c) place the artifact
        exit_code, out, err = await sandbox.exec(       #    (d) run the checker IN
↪  the sandbox
            "python3 run_checks.py", timeout=60)
        if exit_code == 0:                              #    (e) checker passed?
            solved = True
            break
        messages.append(task.observation(out, err))    #    (f) feed the failure
↪  back, loop

    reward = 1.0 if solved else 0.0                    # 3. REWARD (from the checker)
    return build_token_stream(messages, reward)        # 4. loss-masked trajectory ->
     ↪  Sample
```

Walking it: the harness **seeds** the sandbox (step 1), then repeats **ask → parse → run → observe** (step 2a–f), each "ask" going through the proxy to the in-training policy and each "run" going to the AgentCore sandbox. When the checker exits `0` the episode is **solved**; otherwise the failure output is appended and the model tries again, up to `max_turns`. The final **reward** (step 3) comes from the checker — not a judge — and the harness hands the loss-masked trajectory to the training-data path (step 4). The harness owns the loop; the proxy and sandbox are the two services it calls each turn.

2.3 The proxy — every model call is the policy under training

What it is. The proxy is a thin CPU service that exposes a **standard LLM API** (in this project an Anthropic-Messages–compatible `AnthropicAdapter`, but an OpenAI-compatible endpoint works the same) and routes every request to the **in-training inference engine** (SGLang) instead of to any external provider. It is the single

seam that lets an *off-the-shelf agent harness* — one written to call a normal LLM API — drive a model that is still being trained.

What it does. Three jobs, all on the critical path of every turn:

1. **Substitute the endpoint.** The harness thinks it is calling a hosted LLM; the proxy answers with the **policy under training**, holding the latest synced weights. No harness change is needed to point it at the trainer.
2. **Record the exact tokens.** For each call the proxy captures the **prompt token IDs and the sampled response token IDs** (and log-probs). The training engine only ever observes this stream of request/response pairs — so recording the *exact* sampled tokens is what avoids the retokenization/decode-drift problem in harnessed agentic RL [1].
3. **Bucket by episode.** Each request is tagged with the episode's `session_id`, so all calls of one rollout are grouped into the right trajectory before the Sample Adapter turns them into training samples.

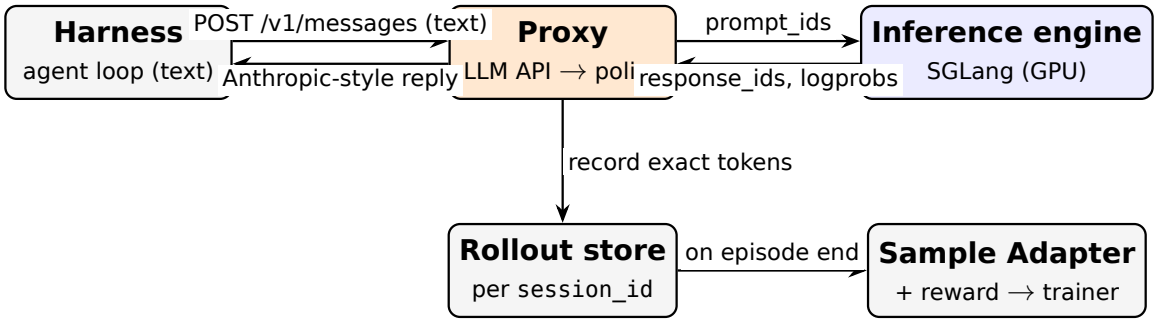


Figure 3: The proxy on the seam between the text-based harness and the token-based inference engine. It **substitutes** the endpoint (the harness's ordinary LLM API call is answered by the policy under training), and it **records** the exact prompt and response token IDs (with log-probs) into a per-`session_id` rollout store. On episode end the Sample Adapter reads that store, attaches the reward, and emits training samples — so the trainer learns from the exact tokens that were sampled — the *token-in-token-out* contract discussed next.

End-to-end example — one turn through the proxy. Consider a single MBPP coding turn:

```
1. Harness builds the chat and calls the proxy (drop-in LLM API):
     POST /v1/messages   { model: "policy",
                           messages: [system, PROBLEM.md, prior observations],
                           metadata: { session_id: "ep_9f3c···" } }
```

```
2. Proxy → Inference engine (SGLang, current policy weights):
     generate(prompt_ids) → response_ids, text, logprobs

3. Proxy records the on-policy call for training:
     rollout_store[session_id].append(
        { prompt_ids, response_ids, logprobs } )   # exact sampled tokens

4. Proxy returns an Anthropic-style response to the harness:
     { content: "```python\n def solve(): ···```", stop_reason: "end_turn" }

5. Harness extracts solution.py, runs it in the AgentCore sandbox,
   appends the checker's stdout/exit-code as the next observation, loops.

6. Episode ends → reward = 1.0/0.0. Sample Adapter reads
   rollout_store["ep_9f3c···"], attaches the reward/advantage, and emits
   loss-masked training samples (mask 0 on prompt/observation tokens,
   1 on the policy's own response tokens).
```

The key point: **without the proxy, the trainer could not observe — or take credit for — the tokens the harness actually sampled.** The proxy is simultaneously the *substitution* point (external API → policy) and the *observation* point (turns → training samples), which is exactly why the disaggregated design in Figure 1 puts it on the seam between the agent runtime and the training server.

2.4 Token-in-token-out (TITO) — the proxy's core responsibility

What it is, and where it comes from. *Token-in-token-out* (TITO) is the contract that the rollout path carries **token IDs**, not just text: the inference engine returns the **exact token IDs it sampled** (plus their log-probs), and the trainer consumes **those same token IDs** for the policy-gradient update — never a sequence re-derived by tokenizing decoded text. The need for it is intrinsic to *harnessed* agentic RL [1]. In the traditional setup the training engine owns the environment loop and keeps one continuously extended token history, so a rollout is a single linear token sequence by construction. In the harnessed setup the **harness** owns context construction, control flow, and tool execution; it builds each prompt independently and talks to the model over an ordinary, **text** LLM API. The trainer never touches the environment — it only observes a sequence of request/response pairs $(p_1, a_1), (p_2, a_2), \ldots$ crossing the LLM endpoint. TITO is the guarantee that what crosses that seam is captured *token-exact*: the a_i the trainer learns from are the very tokens the policy sampled.

The problem — token-prefix continuity breaks (retokenization drift). Harnesses chain calls in **text**: the previous call is a clean text-level prefix of the next prompt. RL, however, needs continuity at the **token** level, and the two do not coincide — a tokenizer is not a clean round-trip, so re-encoding the decoded history can yield a *different* token sequence than the one sampled. The paper isolates three mechanisms:

- **Chat-template non-compositionality.** Rendering a whole message history is not the same as concatenating the renderings of its parts (Template$(A\|B) \neq$ Template$(A)\|$Template(B)): templates insert or drop delimiters at message boundaries. E.g. Qwen's chat template can strip an earlier `<think>` marker.
- **Decode–retokenize drift.** Decoding is not injective, so re-tokenizing the decoded text need not recover the original IDs: the word *having*, sampled as `h+aving`, can re-tokenize to `hav+ing` inside a later prompt — identical text, different token boundaries.
- **Inference-time output transformation.** Tool-call and structured-output handlers parse, normalize, repair, and reserialize a response before returning it, changing whitespace, delimiters, or JSON — so even the *text* folded into the next prompt can differ from what the sampled IDs represent.

If training reconstructs tokens from this text rather than using the recorded IDs, three things silently misalign:

- **The action.** The token the loss is computed on is no longer the token the policy emitted.
- **The log-prob / importance ratio.** On-policy RL weights each token by the ratio of new to old policy probability; if the "old" tokens are reconstructed rather than recorded, the ratio is computed against tokens that were never sampled, biasing the gradient.
- **The loss mask.** The mask that marks *policy* tokens (train) vs. *prompt/observation* tokens (ignore) is defined per token; a shifted boundary leaks environment tokens into the loss, or drops the model's own.

The same drift also breaks *sample merging*: two consecutive calls can be stitched into one training sequence only when their token IDs actually satisfy prefix continuity, so drift turns one rollout into a *dynamic* number of samples. For RLVR — where the reward is a precise 1.0/0.0 from a checker — this token-level slop quietly corrupts the very signal the method depends on.

The solution space. The paper frames four strategies, trading correctness against prefix-computation reuse:

- **Independent per-call training.** Treat each (p_i, a_i) as its own sample; compute loss without assuming any relation to neighbors. Token-exact by construction, but recomputes the long shared prefixes across calls — wasteful.
- **Buffered token replacement** (AReaL, verl Uni-Agent). The proxy buffers each call's text and tokens; when a new request's text matches, it splices the *recorded* response IDs back in so token-prefix continuity always holds. This recovers reuse, but when the buffered IDs differ from those in the actual next request it evaluates the response under a *stitched* prompt the policy never saw — an off-policy discrepancy.
- **Prefix-shared / tree-structured training.** Store common prefixes once and use a branch-aware attention mask so each token attends only to its ancestors. Recovers the reuse of independent training, but demands heavy backend support (tree packing, custom masks/kernels, distributed-gradient handling).
- **Best-effort sequence merging** (Agent Lightning v1.0's choice). Merge two calls only when their observed token IDs satisfy prefix continuity; otherwise close the sequence and start a new one. This preserves the prompt actually consumed during rollout, works with standard dense causal kernels, and lets drift merely lower the merge ratio rather than corrupt the gradient.

Why the proxy owns it. Every one of these strategies rests on the same primitive: *capturing the sampled token IDs at the moment of generation* instead of guessing them from text later. The proxy is the only component that straddles both sides of the seam — it speaks the **text** LLM API to the harness and the **token** API to the inference engine (SGLang returns `response_ids` + log-probs) — so it is the natural, and only correct, place to do it. Concretely the proxy records `{prompt_ids, response_ids, logprobs}` per turn (Figure above), keyed by `session_id` (Agent Lightning's per-call `model_request` event); the **Sample Adapter** then assembles the training `Sample` directly from those recorded IDs and the loss mask and attaches the reward — so the trainer learns from the exact tokens sampled and the whole drift class is sidestepped before any merging decision. The harness stays blissfully text-only, yet the trainer still gets token-exact, on-policy data. In short: **the proxy is where an off-the-shelf, text-based agent loop is reconciled with the token-exact requirement of RL — that reconciliation is TITO.**

2.5 When one episode becomes many samples — subagents, summarization, and advantage assignment

This is a different problem from TITO — its sibling. TITO asks *"are the tokens I train on exactly the ones the policy sampled?"* — a question about a single call chain crossing the text↔token seam. The problem in this subsection asks *"one episode produced how many training samples, and how do I attach reward, advantage, and loss across them?"* — a question about the **shape of a whole rollout**. Both descend from the same root — in *harnessed* agentic RL the harness (not the trainer) owns context construction and control flow, so a rollout is a set of separately-constructed call chains rather than one linear history [1] — but they are handled in different places. Get TITO right and each individual sample is token-exact; get *this* wrong and you train on token-exact samples with **biased rewards and mis-normalized gradients**.

The problem. In traditional agentic RL one rollout is one Markov trajectory and maps to exactly one training sample. In harnessed agentic RL a single episode can fan out into a **dynamic number of training samples** N_ρ, known only *after* the harness has run and samples have been constructed. This is not a corner case: in Agent Lightning's coding-agent runs only **36% of rollouts on average stay a single training sample, and the mean rollout yields 2.4 samples** [1]. Three harness behaviours drive the fan-out — and all three are exactly the ones you called out:

- **Context summarization / compaction.** To stay under the context window the harness periodically replaces the running message history with a summary. That *rewrites the token prefix* the next call is built on, so the new call no longer has the previous call as a token-level prefix — the chain must be cut and a new sample started.
- **Response rewriting / expansion before acting.** The harness may parse, repair, reserialize, or expand a model response (tool-call normalization, JSON repair, template surgery) before it acts on it or folds it into the next prompt — so what re-enters the history differs from the sampled tokens, again breaking the merge condition.
- **Subagents.** The harness spawns a sub-agent that runs its **own** chain of model calls with its **own** prompt history. These branches do not share one linear history with the parent at all; each is its own sample (or its own little tree).

The cause — the harness is the only component that sees a whole episode. The proxy sees a stream of independent per-call (`prompt_ids, response_ids`) pairs; the sandbox sees a stream of independent tool ops and one terminal reward. **Neither sees the episode as a whole** — only the harness, sitting between them, knows that these calls, that summary, and those subagent branches all belong to *one* rollout ρ with *one* outcome. So the harness is the only place that can stamp every fragment with a shared `rollout_id` (and `prompt_group_id`), and the training side must honour that grouping — otherwise it silently treats an incidental fan-out as if it were more independent data. The distinction from TITO matters here: retokenization drift (§"Token-in-token-out") is *incidental* and merely lowers the sample-merge ratio, whereas summarization and subagent spawning are *structural* — they are deliberate harness operations that split a rollout by design.

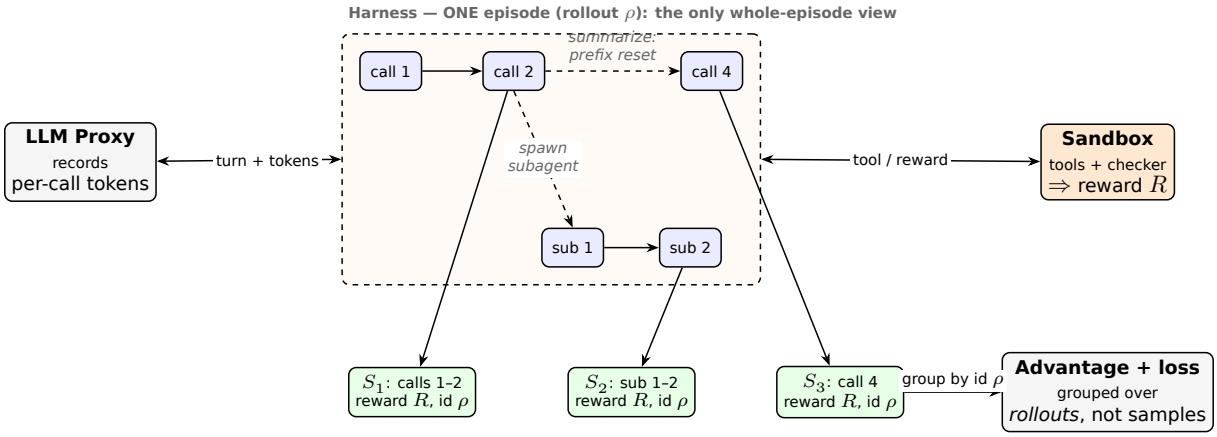


Figure 4: How it works with the other components. The harness sits between the proxy and the sandbox and is the **only** component that sees a whole episode. One rollout ρ fans out into several training samples: a spawned *subagent* runs its own call chain (S_2), and a *summarize* step resets the token prefix so calls 1–2 close as S_1 while the post-summary call 4 opens S_3. Every sample inherits the episode's terminal reward R and carries the same `rollout_id` ρ; advantage baselines and loss normalization must then be computed **per rollout**, not per sample, so an incidental fan-out cannot reweight the update.

Why the naive handling corrupts training. Once one rollout ρ owns N_ρ samples that all inherit its outcome reward R_ρ, two group-level computations become ambiguous:

- **Advantage baseline (GRPO).** GRPO's advantage subtracts a group baseline $\bar{r}$ over the samples generated from one prompt. If the baseline is taken at

the **sample** level, a rollout that happened to split into more samples silently gets more weight in the baseline. Concretely: in one group Rollout 1 succeeds ($R = 1$) and Rollout 2 fails ($R = 0$). If Rollout 1 fanned out into 3 samples and Rollout 2 stayed 1, the *rollout-level* baseline is the intended $\bar{r} = (1+0)/2 = \frac{1}{2}$, but the *sample-level* baseline is $\bar{r} = (1+1+1+0)/4 = \frac{3}{4}$ — a biased baseline that no one chose, driven purely by how the harness happened to fragment. (Frameworks disagree today: verl Uni-Agent and Polar compute advantage at the rollout level; slime and AReaL at the sample level [1].)

- **Loss normalization.** With a dynamic per-rollout sample count, sample-level *seq-mean-token-mean* normalization (GRPO's default) gives disproportionately more gradient weight to rollouts that produced more samples. Since the sample count is an incidental artifact of summarization/subagents, that reweighting is unprincipled.

The solutions — reconcile at the rollout level. The fix is a small, coherent set of choices that make the incidental fan-out invisible to the gradient:

1. **Sample construction: best-effort merging, honest splits.** Merge consecutive calls into one sample only when their token IDs satisfy prefix continuity (the TITO merge condition); when summarization, response rewriting, or a subagent breaks that prefix, **close the current sample and open a new one** rather than force a stitch. Represent the episode as the resulting *set/tree* of samples, all tagged with the shared `rollout_id`.
2. **Reward: outcome-inherited.** The reward is terminal and per-episode (the sandbox checker's $1.0/0.0$); **every sample in the rollout inherits R_ρ.** A subagent's samples carry the parent episode's outcome.
3. **Advantage at the rollout level.** Compute the GRPO group baseline over **rollouts**, not samples, so — as in the worked example — a rollout that fragmented into more samples cannot move the baseline. Agent Lightning argues this is the principled default; retokenization and harness internals "should not be allowed to change the baseline of the entire group" [1].
4. **Loss normalization at the rollout level.** Use a **rollout-level token-mean** loss: pool all response tokens of a rollout, then average uniformly over rollouts (slime's choice). This is insensitive to how many samples a rollout produced. Plain token-mean is theoretically clean too but empirically unstable when a batch has many long negative samples; sample-level seq-mean is the one to avoid here.
5. **Preserve provenance to the backend.** When samples are flattened into

a physical tensor batch, every sequence must retain its `rollout_id` and `prompt_group_id` so steps 3–4 can still be computed after packing onto fixed GPU workers.

6. **Open problem: intra-rollout credit assignment.** Outcome-inheritance is a placeholder — it does not say *which* branch, subagent, or turn earned the reward. Finer credit assignment across the samples within a rollout (turn-level grouping such as GiGPO [26], process rewards) is active research; see the credit-assignment item in §"Open problems in production agentic RL".

Where this project sits. The training demos in this report deliberately use **short, single-chain episodes**: a binary end-of-episode reward with `EPISODE_MAX_TURNS=8`, no context summarization and no subagents, so every rollout collapses to exactly one sample and the fan-out above never bites — which is why `build_token_stream` can safely return a *bare* `Sample` (§"Key code — three pieces"). That is a modelling convenience, not a limitation of the stack. The moment the harness is a real, full-featured agent — one that compacts context and spawns subagents — this subsection's machinery becomes mandatory; the removed `demo_cpu_branching` (§"History: removed demos (2026-09-30)") was a standalone demo that switched the mini_claude harness for the **Claude Code CLI**, deliberately provoked summarization and subagent branching, and was built to show one rollout producing several samples that are then reconciled with rollout-level advantage and rollout-level token-mean loss.

2.6 slime — the trainer, and its asynchronous rollout

The trainer in this report is **slime** (THUDM) — an RL post-training framework built on **Megatron** (training) + **SGLang** (rollout), designed so that agentic workloads plug in as *data-generation* functions without forking the training kernel [9]. Its documented architecture maps cleanly onto Figure 1: a **Data Buffer** bridges a **Megatron** training group and one or more **SGLang** rollout servers behind a **router**; a *custom rollout generation* hook (where our harness + AgentCore sandbox live) produces samples into the buffer, and Megatron periodically pushes updated weights to the SGLang servers.

Asynchronous mode. By default training and rollout alternate, so the GPUs on each side idle while the other runs. slime avoids this with an **async training driver** (`train_async.py` instead of `train.py`): while Megatron

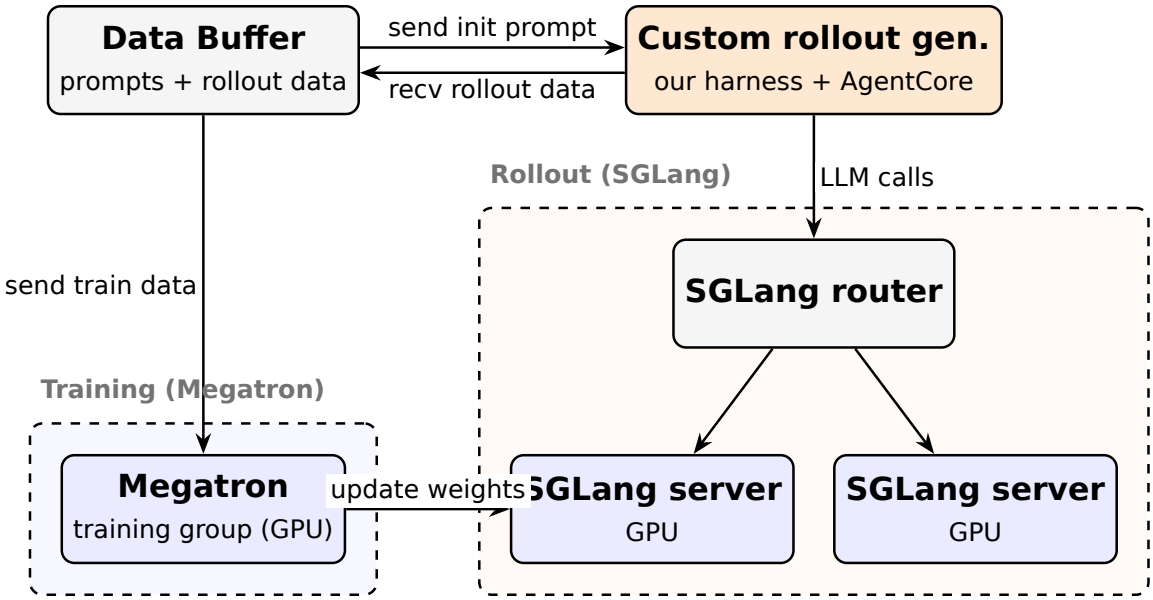


Figure 5: slime's architecture (redrawn after the slime documentation [9]). A **Data Buffer** bridges the two sides: it hands prompts to a *custom rollout generation* hook — where our **harness + AgentCore sandbox** live — and receives finished rollout data back. Generation runs through an **SGLang router** to one or more GPU **SGLang servers**. On the training side the buffer feeds **Megatron**, which periodically pushes **updated weights** to the SGLang servers. Agentic workloads plug into the hook without forking the training kernel.

trains on the current batch, the rollout side already generates the *next* rollout's data, overlapping the two [9]. For agentic RL — where rollouts have a **long tail** (one slow episode, e.g. a lengthy compile, can stall a whole step) — slime also offers a **fully-async rollout** path (`--rollout-function-path slime.rollout.fully_async_rollout.generate_rollout_fully_async`) that keeps a fixed pool of in-flight generations across rollout boundaries, so a training step never waits on the slowest sample. This is exactly the regime this report targets: **AgentCore sandboxes have variable, sometimes long, per-episode latency**, and async rollout keeps the trainer fed while they run.

Sources: slime repository https://github.com/THUDM/slime and documentation https://thudm.github.io/slime/ [9].

2.7 Scaling the loop — inference, harness, and sandbox in balance

The rollout loop has **three** components, and they scale as one system because they are interconnected. The **inference engine** (SGLang) produces every model turn; the **sandbox** (AgentCore) runs every tool call and the checker; and the **harness** — the agent-loop driver that opens each episode, calls the policy turn by turn, runs the checker, and packs the result into a `Sample` — sits between them and paces both. Rollout throughput is bounded by the slowest of the three: **buffer-fill rate = `min(inference, sandbox, harness)`.** Scale two and leave the third and the bottleneck merely moves onto the third. So the job has three parts, in order: **diagnose** which tier is slow, **scale** it, and **balance load correctly on each tier** — the three balance by *different* rules, and applying one tier's rule to another is a mistake. This subsection treats all three together; the diagram below is the map.

Why scale the sandbox at all? A policy only *generalizes* if it is trained across many **diverse tasks** — SWE repos, SQL, math, tool-use, RAG, CTF, GPU kernels — and across **many episodes of each** so the reward signal is not dominated by one family. That means running a large number of **concurrent episodes** per training step, and every concurrent episode needs its own isolated sandbox. So scaling *up the number of concurrent sandboxes* (the admission knob `CONC`: `S = CONC ×` engines episodes in flight) is the direct lever on task diversity and throughput.

But sandboxes do not run in isolation. Every one of those concurrent episodes still gets its model turns from the **same** inference engine. If we raise sandbox concurrency while holding the number (and size) of inference replicas **fixed**, generation

requests start to **queue** at the inference engine: per-turn latency climbs, and the extra sandboxes sit idle waiting for their next turn. Past that point the inference engine — not the sandbox — is the bottleneck, and adding sandboxes buys nothing.

So first diagnose which side is slow. Per episode the wall-clock is split between (a) **generation** — the inference engine producing the turn — and (b) **sandbox execution** — running the tool call and the checker. The harness measures both: `[sglang-gen]` → `perf/sglang_gen_ms_{p50,p95}` for generation, and `[sbx-op]` → `perf/sandbox_ms_{p50,p95}` for the sandbox round-trip. Compare them before scaling:

- **Generation dominates** (`sglang_gen_ms` ≫ `sandbox_ms`) ⇒ **inference-bound**. Scale the inference engine (more replicas / more GPUs / larger tensor-parallel), *not* the sandbox count.
- **Sandbox dominates** (`sandbox_ms` ≫ `sglang_gen_ms`) ⇒ **sandbox-bound**. Scale sandbox concurrency / instances; one inference replica can feed many episodes.

Examples.

- **Sandbox-bound — KernelBench (GPU).** The model emits a short CUDA kernel in a few hundred ms, but `nvcc` compilation plus `allclose` on the GPU takes **seconds**. Sandbox time dwarfs generation → add GPU sandbox Instances; a single inference replica keeps up.
- **Inference-bound — math / long-reasoning.** The checker is a one-line stdlib assert (sandbox ≈ tens of ms), but the model generates a long chain of thought (generation ≈ seconds). Inference time dwarfs the sandbox → adding sandboxes does nothing; add inference replicas.
- **Balanced — SWE with a moderate test suite.** Generation and a multi-second test run are comparable → scale **both** roughly in proportion to keep either side from queuing.

The practical loop: pick a concurrency, read `perf/sglang_gen_ms_*` and `perf/sandbox_ms_*`, and grow whichever side dominates until the two are balanced — then raise both together.

Balancing load as you scale — least-loaded engine per episode, round-robin the sandbox pool. Once you add replicas on *either* side, the next question is how episodes get spread across them. The two sides answer it differently, and it matters that you *not* apply the same rule to both:

> **Do you need a load balancer?** For the **sandbox: no** within one Runtime — the session id *is* the balancer (the capacity provider places each session for you); across a *pool* of Runtime ARNs you add only trivial **sticky round-robin by session id**, not a real LB. For **inference: also no** — the harness tier (`harness_pool.py`) reads the engine list from slime's SGLang router once and sends each *episode* to the engine with the fewest in-flight episodes, sticky for all its turns. In short: you never stand up a load balancer of your own; you mint distinct session ids, count in-flight episodes per engine, and (only across a Runtime pool) route sessions stickily.

- **Inference — balance per episode, then stay sticky.** slime runs **one SGLang router** in front of its engines, which balances each *request* cache-aware (prefix affinity, shortest-queue fallback). A multi-turn episode's prompt grows turn by turn on its own prefix, so the harness tier balances one level up: it sends each **episode** to the engine with the fewest in-flight episodes (ties rotate, so an idle start does not pile onto engine 0) and keeps **all of that episode's turns** there, where its prefix is already in the KV cache. This replaced an earlier crc32 hash of the episode onto an engine (`router_trace.pin_engine`), which was sticky but blind to load. What to avoid is per-request round-robin: it would scatter an episode's prefix across every replica and destroy the prefix cache. Scaling inference stays "add engines"; the per-episode `engine_inflight` field shows when inference binds.
- **Sandbox — the session id *is* the load-balancing key.** One `runtimeSessionId` maps to exactly one isolated microVM / instance. Within a **single** Runtime, AgentCore's capacity provider already **places each new session on an available instance**, so simply minting a distinct per-episode session id spreads the load for you. You only need *explicit* round-robin when you run a **pool of sandbox Runtimes** (several ARNs). Three things trigger a pool — otherwise a single ARN is enough: **(1) concurrency** — your in-flight episodes (`S = CONC × num_engines`) would exceed one Runtime's per-region session cap, so you split the load across K ARNs; **(2) AZ/account spread** — placing Runtimes in different availability zones (resilience) or accounts (isolation, separate quotas); **(3) capability routing** — different families need *different sandbox images* (a GPU family, a DB-baked image, a per-repo SWE image, §3), so you register one ARN per capability and pick by `family` rather than pure round-robin. Below these thresholds — episode count fits one Runtime's quota and one generic image serves every family — you stay on a **single ARN** and let the capacity provider place sessions for you. When you do pool, pin each

episode to one ARN and make it **sticky** so every op of that episode returns to the same warm sandbox that holds its workspace:

```python
import hashlib, os

# 1) Inference: the least-loaded engine per EPISODE, sticky for all its turns
#    (harness_pool.py). The engine URLs come from slime's router; ties rotate.
def acquire_engine(self):
    n = len(self._inflight)
    self._rr = (self._rr + 1) % n
    i = min(range(n), key=lambda j: (self._inflight[j], (j - self._rr) % n))
    self._inflight[i] += 1                    # released when the episode ends
    return i, self._engine_urls[i]

# 2) Sandbox: within ONE Runtime the session id already spreads sessions across
#    instances. Across a POOL of sandbox Runtimes, round-robin BY session id so
#    the spread is even AND sticky (all ops of an episode hit one warm microVM).
SANDBOX_ARNS = os.environ["AGENTCORE_SANDBOX_RUNTIME_ARNS"].split(",")

def pick_sandbox_arn(session_id: str) -> str:
    h = int(hashlib.sha1(session_id.encode()).hexdigest(), 16)
    return SANDBOX_ARNS[h % len(SANDBOX_ARNS)]   # deterministic, sticky per episode
```

Hashing the session id (rather than a shared counter) gives an even, **sticky** spread with no cross-thread lock — which matters because episodes run concurrently on `asyncio.to_thread`. A plain counter (`itertools.cycle` behind a lock) is the simpler *literal* round-robin, but only use it if you do not need stickiness — and here you do: every op of an episode (seed → exec → read) must land on the microVM that holds that episode's files.

*(Implementation note: the shipped `make_sandbox()` (`sandbox.py`) actually uses the **`itertools.cycle`-behind-a-lock** counter, not the session-id hash sketched above. Stickiness still holds — the ARN is chosen **once** when the episode's sandbox object is created and fixed for all its ops — so each episode keeps one warm microVM; only the* selection* is a rotating counter rather than a hash. The hash form above is the lock-free alternative you would switch to if the shared counter's lock became a contention point at high S.)*

The third tier — the harness, the driver behind both dials. The two dials above, inference and sandbox, are both **driven** by a third component that is easy to forget because it runs neither on a GPU nor behind a sandbox API: the **harness** — the agent loop that seeds each episode's sandbox, calls the policy turn by turn, runs the tools and the checker, and packs the result into a `Sample`. It has its own

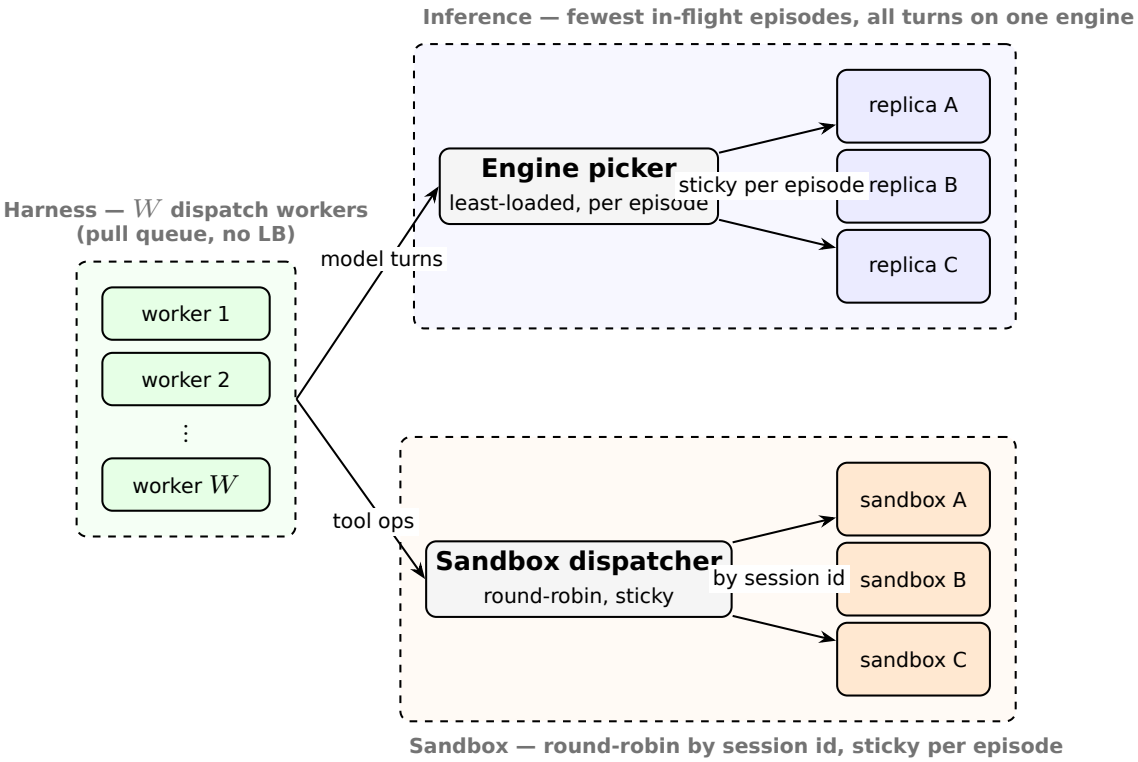


Figure 6: The three interconnected tiers scale together, and **each balances load by a different rule** — buffer-fill rate is `min(inference, harness, sandbox)`. **Harness (left):** W = `HARNESS_WORKERS` dispatch workers in the trainer (default one rollout step, `RB` $\times$ `NS`); they need *no* load balancer — a free worker *pulls* the next episode off one shared queue (work-stealing). The agent loop itself runs *inside* the episode's sandbox session, so the harness scales with the sandbox, not the trainer. **Inference (top):** each episode takes the SGLang engine with the fewest in-flight episodes (ties rotate) and keeps all its turns there, so its growing prompt prefix stays in one KV cache; scaling is "add engines." **Sandbox (bottom):** one `runtimeSessionId` = one isolated microVM/instance, taken *before* the engine. Within a single Runtime the capacity provider spreads sessions for you; across a *pool* of sandbox Runtimes you **round-robin by session id**, kept **sticky** so every op of an episode (seed $\rightarrow$ exec $\rightarrow$ read) returns to the same warm microVM that holds its files.

throughput ceiling, and if you scale inference and sandbox to fill a deep async buffer while leaving the harness alone, the bottleneck simply **moves onto the driver**: the buffer-fill rate is `min(inference, sandbox, harness)`, and adding replicas to two of the three buys nothing once the third saturates. So the harness has to scale **along with** SGLang and the sandbox — otherwise a few coroutines silently become the throughput ceiling.

Where the old ceiling was: one process. slime runs **one** `RolloutManager` Ray actor whose single event loop drives *every* `generate` coroutine (`slime/ray/rollout.py`); the number of concurrent episodes is bounded by one `asyncio.Semaphore` sized `S = CONC × num_engines` — with the demos' `CONC=4` and two SGLang engines, `S = 8` in flight (`slime/rollout/sglang_rollout.py`). There is **no slime knob for "number of rollout worker processes"**: `CONC` raises concurrency *within* the one process, not the number of processes. In the original design every episode's agent loop ran on that one event loop, next to the trainer.

Why one process ceilings. Concurrency there is `asyncio`, so it overlaps *I/O waits* — but not CPU work, which the Python **GIL** serializes: every turn JSON-encodes an invoke payload, decodes the response, and (at episode end) builds the loss-masked token stream, and those run one-at-a-time. The synchronous boto3 `invoke_agent_runtime` is offloaded with `asyncio.to_thread` onto the loop's default executor, which is ≈ `min(32, cpu+4)` threads unless configured, and every in-flight episode also holds open file descriptors / sockets and buffered trajectory state in RAM. So past some `S`, raising `CONC` just deepens internal queues: per-episode latency climbs and the buffer stops filling faster.

What the harness tier changes — the default in every demo. The fix has two halves (`harness_pool.py`, the default rollout topology of every demo since 2026-09-28, like `PREOPEN` and `HEARTBEAT`). **(1) The loop moves into the sandbox session.** The mini_claude loop runs inside the episode's own session and calls the engine's adapter back, so N sessions carry N loops and the harness scales **with the sandbox, not as part of the trainer** — the placement DSec uses [31]. **(2) What stays in the trainer is a dispatch tier sized on its own:** `W = HARNESS_WORKERS` workers (default `RB × NS`, one rollout step) pulling episodes off one shared queue, one `AnthropicAdapter` per engine, the loop's default executor sized to the pool (`max(W, M) + 32` threads instead of the stock 32), and one warm `aiohttp` session with no connection limit. The three tiers are sized independently and joined **per episode** — no worker owns an engine or a session: a free worker pulls the episode, takes

its sandbox session first (fresh by default; `SANDBOX_SESSIONS=M` borrows one of M pooled sessions per image, in a fresh workdir), then the least-loaded engine. Each episode's `[demo]` record names the tier that binds: `qdepth` > 0 = harness-bound, `sandbox_wait_ms` > 0 = sandbox-bound, climbing `engine_inflight` = inference-bound.

Component	Lifetime / isolation	Scaling unit	Ceiling of one unit	Where it runs
Inference (SGLang)	persistent / shared	replica behind the router	GPU KV cache + decode throughput	trainer GPUs
Harness	**the loop: per episode, in its sandbox session; the dispatch tier: persistent**	**one loop per session; `W` dispatch workers**	**trainer side: the adapters' tokenisation and trajectory flattening (GIL)**	**the loop in the sandbox session; dispatch + adapters on the trainer box**
Sandbox (Agent-Core)	ad-hoc, per episode / strong (microVM)	session id; pooled Runtime ARNs	per-Runtime session cap	AgentCore microVM / Instance

Where to put the harness — three options.

- **Co-located on the trainer box (the original design).** The one `Rollout-Manager` runs every loop next to Megatron and SGLang. Cheapest and lowest-latency to the buffer, and fine at $S \approx 8$; but the loops' threads, FDs, and RAM compete with the trainer for the box's CPU budget, and the one event loop and the GIL become the ceiling as you push S up.
- **Scaled out to a CPU rollout fleet (disaggregated).** Because slime drives generation from a single actor, going past one process is an *architecture* change, not a flag flip: run several rollout drivers (shard prompts across CPU nodes, each executing the same `generate` hook) that dial the **shared** SGLang router and the **pooled** sandbox ARNs. This decouples harness

CPU/FDs/RAM from the GPU box and lets you scale drivers independently — the disaggregated-placement lever of §“Open problems” applied to the harness rather than to the trainer/inference split.

- **Inside the sandbox session (the default in every demo since 2026-09-28).** The loop runs where its tools run, one per session, and dials the trainer’s per-engine adapter back over the VPC (the adapter ports are admitted SG→SG). The earlier alternative, a separate pool of harness Runtimes (the legacy two-runtime `agentcore_runtime_generate.py / harness_runtime_agent.py`, now deleted), added a second fleet to size and a network hop; running the loop in the episode’s own session gets the loop out of the trainer without a second fleet. There is no in-trainer fallback: a session that cannot host the loop fails the episode with `HarnessPlacementError`.

The joint recipe. Diagnose the harness the way §“Scaling the loop” diagnoses the other two, from the per-episode `[demo]` fields: `qdepth` > 0 means episodes wait for a free dispatch worker (raise `HARNESS_WORKERS`), `sandbox_wait_ms` > 0 means they wait for a session (raise sessions, the Runtime pool or `SANDBOX_SESSIONS`), and a climbing `engine_inflight` means inference binds (add engines). Then grow the three in balance. A Little’s-law sizing keeps them matched: the in-flight episodes needed to saturate inference ≈ `inference throughput × mean episode latency`; provision `S = CONC × engines` and `W` to hit that number (the removed `SCALE=1` preset did exactly this at $4 : 256 : 256$; see §“History: removed demos (2026-09-30)”), then raise all three together until GPU **goodput** — not any single dial — stops improving.

3 Tasks frontier labs run in a sandbox

Frontier models are post-trained with **reinforcement learning against executable, verifiable environments** (RLVR). The common denominator is a **sandbox seam**: the model’s untrusted output runs in an isolated environment, and a **programmatic verifier** — not a human, and increasingly not an LLM judge — turns the outcome into a scalar reward.

3.1 What the frontier reports describe

The trend is toward **verifiable rewards at scale**: Kimi K2, for example, runs a rollout infrastructure of **more than 10,000 concurrent execution sandboxes** on Kubernetes for its coding and software-engineering RL [5]. What the major open reports describe:

Model	RL / verifier setup	Task domains	Reward
DeepSeek-R1 [2]	GRPO; rule-based verifier + code compiler (no neural reward model, to avoid reward hacking)	math, competitive code	answer match / tests pass (binary)
Qwen3 [3]	Reasoning RL from **3,995 query–verifier pairs**; General RL over **20+ tasks**; multi-turn tool use with real-environment feedback	math, code, tool-use, RAG, instruction	rule-based verifiable + reference-based model judge + reward model
Llama-Nemotron [4]	GRPO (science) + RLOO (instruction following); vLLM + Megatron; prompts difficulty-filtered by pass-rate	science reasoning, math, instruction	**LLM-judge** (Llama-3.3-70B) answer-match + `<think>` format reward
AceReason-Nemotron [10]	math-then-code RL; a **local sandbox verifier** runs code against *all* test cases (64-worker pool)	math (~49k problems), code (~8.5k problems)	binary: pass **all** tests / `sympy` symbolic match on `\boxed{}`

Model	RL / verifier setup	Task domains	Reward
Kimi K2 [5]	agentic RL; **verifiable-rewards "gym" + a tool simulator (world model) + real execution sandboxes** (>10k concurrent)	agentic tool use, coding, SWE, math, instruction	executable unit tests + verifiable checks + self-critique rubric

NVIDIA also **publishes the post-training data mix** openly: the Nemotron post-training datasets [12] quantify the domain balance used to train an open reasoning model — roughly **Math 239k, Code 175k, STEM 355k, Chat 628k** samples (plus large multilingual splits) — a concrete picture of how much of each family a frontier post-training run consumes.

3.2 Task families in detail

Each family below lists its **dataset, reward function, key metric, and environment requirement** — the properties that determine what a sandbox must provide — plus an **episode shape**: the typical *turns per episode*, the *response length per turn* (the policy's own tokens), and the resulting *total episode length* (all turns' responses plus the observations fed back). The episode shape is what you size the training config against: it sets **`--rollout-max-response-len`** (per-turn generation budget), the **max turns** cap (`EPISODE_MAX_TURNS`, default **3** in this project's demos), the **sequence-length / context window** the trainer and inference engine must hold, and the **rollout-buffer** memory per sample. The numbers below are typical order-of-magnitude ranges (reasoning models sit at the high end of every response figure), not hard limits — measure your own before committing a config.

3.2.1 Single-turn, generation-bound families

The checker is milliseconds; nearly all the wall-clock is one long generation. These are the *inference-bound* families — scale the SGLang side.

- **Math & logical reasoning** — *Dataset:* GSM8K, MATH-500 [2], AIME / HMMT; **DeepScaleR + NuminaMath** for RL (~49k problems) [10]. *Reward:* **verifiable answer match** — e.g. a `sympy`-based check of symbolic equivalence on the `\boxed{}` answer, binary, with no format or length bonus [10]. *Metric:* accuracy, pass@k (Qwen3-235B's AIME'24 rose **70.1 → 85.1** over 170 RL steps [3]). *Env:* a lightweight **CPU checker** (milliseconds). *Episode shape:* **almost always single-turn** — there is no environment feedback loop, just one answer. But that one response is a **long reasoning trace: ~1k–8k tokens, and up to 16k–32k** for hard competition problems on a reasoning model. Episode length ≈ that one response; strongly **inference-bound** (the checker is a one-line assert).
- **Code generation & competitive programming** — *Dataset:* MBPP [8], HumanEval, **LiveCodeBench** [11]; competitive sets from **Codeforces, AtCoder, LeetCode** [10]. *Reward:* **hidden unit tests pass** — AceReason grants reward *only if the code passes every test case* [10]. *Metric:* pass@1 / pass@k, Codeforces ELO, LiveCodeBench (Kimi K2 v6: **53.7%** [5]). *Env:* a language interpreter plus a **test-runner sandbox with per-case time limits**, run at scale (AceReason: a 64-worker process pool [10]; Kimi K2: >10k concurrent [5]). *Episode shape:* **few turns — often 1, up to ~5** with retry on failed tests. The submitted program is short (**~100–800 tokens**), but a *reasoning* model prepends a long chain-of-thought, pushing the per-turn response to **several thousand tokens**; total episode is typically **~2k–16k tokens**, generation-heavy rather than context-heavy.
- **Instruction following** — *Dataset:* synthetic prompts carrying **one to ten verifiable instructions** (Llama-Nemotron's RLOO stage [4]; part of Qwen3's 20+-task General RL [3]). *Reward:* **deterministic verification via a code interpreter** for the checkable constraints, plus a **format reward**, with an LLM judge for the open-ended remainder [3, 4]. *Metric:* IFEval-style constraint satisfaction. *Env:* a **code interpreter** to evaluate the constraints. *Episode shape:* **typically single-turn** — one response scored against the constraints. Response length tracks the asked-for output, **~200 tokens–2k**; episode ≈ that one response.

3.2.2 Iterative, few-turn families

A short retry loop against a checker — write, run, read the failure, fix. One family (GPU kernels) is where the *checker* dominates cost.

- **GPU kernels / systems** — *Dataset:* KernelBench [7]. *Reward:* numerical **`allclose`** vs. a reference **plus a speedup target**. *Metric:* correctness rate, speedup. *Env:* an **NVIDIA GPU + CUDA toolchain** (`nvcc`) — the one family that requires a GPU, not just CPU. *Episode shape:* **a few turns (1–5), iterating on compile errors and speedup. Each response is one CUDA kernel, 200–1,000 tokens**; total episode a **few thousand tokens**. Cost lives in the *sandbox* (`nvcc` + `allclose` take seconds), not in generation — the canonical sandbox-bound family.
- **SQL / data analysis** — *Dataset:* text-to-SQL suites (Spider-style). *Reward:* **result-set equality** against gold rows (the query is executed against a real engine). *Metric:* execution accuracy. *Env:* a local SQL engine (e.g. SQLite), self-contained. *Episode shape:* **1–3 turns** (write query; retry on a SQL error or empty result-set). Response is a single query, **~30–300 tokens**; episode is **small (a few hundred to ~2k tokens)** — cheap on both sides.

3.2.3 Multi-turn, observation-heavy families

The response each turn is short, but the sandbox feeds a large observation (tool result, retrieved passages, command output) back every turn — cost moves into *context growth and the sandbox*.

- **Tool use / function calling** — *Dataset:* live and **simulated** tool environments; Kimi K2 builds a repository of **3,000+ real MCP tools plus 20,000+ synthetic tools** and generates multi-turn trajectories against a **tool simulator that acts as a world model** [5]; Qwen3 trains with real-environment execution feedback [3]. *Reward:* **correct final result *and* the right tools invoked** (verifiable), sometimes with a model judge. *Metric:* task success on agentic tool-use benchmarks (Kimi K2: **τ^2-Bench 66.1, ACEBench 76.5** [5]). *Env:* tool stubs / an API sandbox / a **stateful simulator** that returns realistic feedback across turns. *Episode shape:* **multi-turn, one turn per tool call — typically ~3–10+ turns**. The policy's response each turn is short (a tool call plus brief reasoning, **~50–300 tokens**), but the *observations* (tool results)

can be large; total episode is **~2k–10k tokens**, dominated by fed-back tool output rather than by generation.

- **RAG & search agents** — *Dataset:* open-domain QA over a corpus or live web search (part of Qwen3's General RL, which explicitly targets RAG [3]). *Reward:* **answer grounded in retrieved facts** — Kimi K2 adds a **sentence-level faithfulness judge** [5]. *Metric:* exact-match / F1. *Env:* a retrieval index or search API plus **network egress**. *Episode shape:* **multi-turn — ~2–8 turns** (issue query → read passages → refine or answer). The policy's responses (queries, final answer) are short, but each retrieved-document observation is large, so the **episode context is retrieval-heavy — commonly 8k–32k+ tokens** even though the model generates comparatively little.
- **Security / CTF** — *Dataset:* CTF challenge sets. *Reward:* a **checker validates the captured flag**. *Metric:* solve rate. *Env:* an isolated shell, sometimes with network to a remote challenge service. *Episode shape:* **multi-turn exploration — ~5–30+ turns** of shell commands. Per-turn responses are short (a command plus reasoning, **~50–400 tokens**); observations (command output) vary from a line to a hex dump, so the **episode ranges from a few thousand to tens of thousands of tokens**.

3.2.4 Long-horizon, session-bound families

Tens to hundreds of turns; the accumulated transcript is the largest of all, and the session must stay alive for the whole workflow — the families that most need a warm, long-lived sandbox.

- **Software engineering (SWE)** — *Dataset:* SWE-bench / SWE-bench Verified [6]; Kimi K2 additionally mines **executable unit tests from real GitHub pull requests and issues** [5]. *Reward:* the repo's **test suite turns green** (`FAIL_TO_PASS` + `PASS_TO_PASS`), binary. *Metric:* % issues resolved (Kimi K2 reports **65.8%** single-attempt on SWE-bench Verified [5]). *Env:* a per-repository toolchain — usually a **prebuilt Docker image** carrying the exact dependencies, run in isolation per task. *Episode shape:* **long-horizon and multi-turn** — real SWE agents take **~15–50+ turns** (locate → edit → run tests → read failure → re-edit); each turn's response is a command or a **diff/patch of ~100–1,000 tokens**, but the observations (file contents, test logs) are large, so the **episode context routinely reaches tens of thousands of tokens (16k–128k+)** — the most demanding family for context length.

- **Web / computer use** — *Dataset:* WebArena-style browser tasks. *Reward:* **goal state reached** (a DOM / state checker). *Metric:* task success. *Env:* a real **browser / rendering engine** and network. *Episode shape:* **long-horizon — ~10–50+ turns**, one UI action (click/type/scroll) per turn. Each response is tiny (**~20–150 tokens**), but every observation (a DOM dump or accessibility tree, sometimes a screenshot) is very large, making this — with SWE — the **heaviest family for episode context (tens of thousands of tokens, growing per turn)**.
- **Multi-agent / long-horizon** — *Dataset:* cooperative or long-workflow tasks. *Reward:* a **verifiable end state**. *Metric:* task success. *Env:* many turns / long wall-clock, with state persisted across a long session. *Episode shape:* **the longest horizon — tens to hundreds of turns**, per-turn responses short-to-moderate, but the accumulated transcript makes the **episode the largest of all (often 32k–128k+ tokens)** and demands a session that stays alive for the whole workflow.

3.3 Language and environment per family — and how one microVM serves them all

The families above differ sharply in *what must be installed to run and check them*. The table names, per family, the **language the model's artifact is written in**, the **environment dependencies the checker needs**, and whether it fits the **CPU microVM** or requires the **GPU Instances** substrate. This is the concrete "what runs inside" that the sandbox image (or a per-session install) must provide.

Family (§3)	Dataset(s)	Artifact language	Environment / dependencies the checker needs	Substrate
Math & logical reasoning	GSM8K, MATH-500, DeepScaleR	Python (answer string)	Python stdlib + `sympy` (symbolic/numeric match)	CPU

Family (§3)	Dataset(s)	Artifact language	Environment / dependencies the checker needs	Substrate
Code gen & competitive	MBPP, HumanEval, Live-CodeBench	Python (competitive: often C++)	Python interpreter + test runner (`unittest`/asserts) + per-case timeout	CPU
Instruction following	synthetic (IFEval-style)	text / Python	code interpreter (stdlib) to verify each constraint	CPU
GPU kernels / systems	KernelBench	**CUDA C++** + Python driver	**NVIDIA GPU + CUDA toolkit (`nvcc`) + PyTorch** (`allclose`, speedup)	**GPU**
SQL / data analysis	Spider-style, Gretel text-to-SQL	SQL (+ Python harness)	a SQL engine — `sqlite3` (in the Python stdlib)	CPU
Tool use / function calling	xLAM, ToolACE	Python / JSON tool calls	Python + tool stubs / API sandbox / a **stateful simulator**; MCP via Gateway	CPU

Family (§3)	Dataset(s)	Artifact language	Environment / dependencies the checker needs	Substrate
RAG & search agents	HotpotQA, open-domain QA	Python (queries + answer)	a retrieval index / search API + **network egress**	CPU
Security / CTF	CTF challenge sets	shell + C / Python / binaries	an isolated shell + common tooling, sometimes **network** to a challenge service	CPU
Software engineering (SWE)	SWE-bench / Verified	Python (repo language varies)	**per-repo toolchain** — pinned deps + system packages, `pytest`; a large image	CPU (amd64)
Web / computer use	WebArena-style	Python + browser actions	a **headless browser / rendering engine** + network	CPU
Multi-agent / long-horizon	cooperative / long-workflow	varies	the sub-tasks' deps + a **long-lived session** holding state	CPU

How can one microVM support such different environments? The key is that **the microVM does not need one universal image** — it reconciles the variation three ways, in increasing order of how structural the difference is (this is the §"One image per Runtime, but a different env per row" logic applied to *languages/deps* rather than to per-repo images):

1. **Task-agnostic sandbox → most differences are just *files*, not images.** The sandbox exposes only `write_file` / exec / `read_file`, and every session is an **isolated microVM** booting *your* OCI image. All task specificity — the prompt, seed files, and the hidden checker — is *written into that episode's session at seed time*, not baked into the image. So a **single generic Python image** already serves the CPU families that need nothing beyond the interpreter and stdlib: math (`sympy`), code (`unittest`), SQL (`sqlite3` is stdlib), instruction following, tool-use stubs, and light CTF. A math episode, a SQL episode, and a tool-use episode are just *different files* in three identical, isolated microVMs.
2. **Per-session runtime install → heavier or unusual deps go into the ephemeral workspace.** This has two variants with *different* implementation status. **(a) Deferred to `CMD`** — the sandbox image installs a large dep when the container boots, before the server binds, keeping the *built* image small; this is **implemented** and is how the removed GPU demos' sandbox installed the ~2.7 GB CUDA torch wheel to stay under the 2 GB image cap. **(b) Installed at *seed time*** — the harness would `git clone` / `pip install` / `apt` a family's deps into that session's workspace on turn 0. This is a **described design, not yet implemented**: the shipped demos seed only small files (`PROBLEM.md`, `run_checks.py`, `solution.py`), and the seed-time-install path is the SWE workaround the report explicitly flags as *"a design answer, not a measured one"* (§"One image per Runtime, but a different env per row"). Either way, because each `runtimeSessionId` is a **separate microVM**, one episode's installed environment **never leaks into another's** — the isolation is what would make per-session customization safe.
3. **Per-capability image → a *pool* of Runtimes when the difference is structural.** Some differences cannot be installed at runtime: a **GPU toolchain** (KernelBench needs `nvcc` + a GPU — the **Instances/amd64** substrate, not the CPU microVM), a *pinned OS image* per task (Terminal-Bench), root `apt` the base image can't grant, or an image over the 2 GB microVM cap. For those you register **one Runtime ARN per image**, and routing is now **implemented**: every task carries an `image` tag, and `make_sandbox(image=⋯)` → `_runtime_arn_for_image` looks it up in `AGENT-`

CORE_SANDBOX_RUNTIME_ARN_BY_IMAGE (falling back to round-robin over AGENTCORE_SANDBOX_RUNTIME_ARNS). AgentCore itself never sees the task — the routing is entirely client-side (the Terminal-Bench demo's "Strict routing" design point walks the chain, §"The Terminal-Bench demo").

In short: **most task diversity needs no task-specific image.** The managed AgentCore **Code Interpreter** (the training default until 2026-09-28, a probe since) already ships Python 3.12 with numpy, pandas, scipy, scikit-learn, sympy, torch (CPU), sqlite3, gcc and node, and a live probe (2026-09-25) passed **all 13 task families** at reward 1.0 on it (#1); since the agent loop moved into the sandbox session, the demos run the same tasks on one generic own-Runtime image instead, because the loop must reach the trainer's adapter. Only genuine image/substrate differences — a pinned per-task OS (Terminal-Bench), an image over 2 GB, git/internet-dependent installs, or CUDA — need your own Runtime (#3); seed-time installs (#2b) remain a design for SWE workloads, not a path the current demos exercise.

3.4 The one family that stresses the image: SWE and the microVM's arm64 + 2 GB limits

Most families above run inside a small, generic image — a Python interpreter and a stdlib checker fit comfortably. **Software engineering (SWE)** is the exception, and it is worth calling out before the platform section because it collides with two properties of AgentCore's *managed serverless microVM* — and the two collide with each other:

- **arm64-only (documented).** The managed microVM runs **only arm64** images: the official Runtime service contract states the *"Platform: ARM64 container — Required for compatibility with the AgentCore Runtime environment"* [D17] (the GPU capacity-provider substrate, by contrast, is amd64 — §"Building a customized Runtime"). A per-repository SWE image must therefore be **cross-built to arm64**.
- **A tight built-image ceiling.** The managed microVM image must stay small — this project measured a **~2 GB (2048 MB)** ceiling on the built image, which is exactly why its GPU sandbox installs the CUDA torch wheel at CMD rather than baking it in (§"Building a customized Runtime"). A realistic SWE-bench repo image — the exact compiler, system packages, and pinned dependencies a project needs — routinely exceeds a couple of GB.

The two interact badly for SWE specifically, because **SWE-bench's own tooling treats arm64 as second-class**: its README states plainly that *"Support for `arm64` machines is experimental"*, and on ARM hosts it does not ship prebuilt images — you must pass `--task-repo` so *"the images are built locally with Docker Buildx"* [6b]. So the very images you would reuse are (a) only *experimentally* available on the architecture the microVM forces, and (b) sized for x86 developer boxes, not a 2 GB serverless cap. The heaviest-image family lands on the substrate least suited to it.

Workarounds — in rough order of how well they fit this project's architecture:

1. **Fetch the repo and deps at *seed* time, not in the image.** The sandbox is task-agnostic (`write_file` / `exec` / `read_file`), so the harness can `git clone` the target repo and `pip install`/build its dependencies into the *session workspace* on the first turn, instead of shipping them in the OCI image. This moves the repo bulk off the image entirely and into the per-session ephemeral filesystem (good for 8 h) — sidestepping the image cap. Cost: per-episode setup latency and **network egress**, so the Runtime needs public egress or VPC interface endpoints (§"Scaling, sharing, and network isolation"; [D11]).
2. **Defer heavy installs to `CMD`.** Keep the *built* image lean and `pip`/`apt` the large layers when the session boots — the trick the removed GPU sandbox used for torch (§"Building a customized Runtime"). Cost: added cold-start (the removed GPU demo paid ~90 s for its torch install).
3. **Slim the repo image.** Multi-stage builds, `--no-install-recommends`, drop build toolchains and package caches after compiling, and start from a slim arm64 base — standard Docker hygiene, often enough to bring a repo image under the cap.
4. **Cross-build to arm64 with Buildx, and *validate the checker there*.** `docker buildx build --platform linux/arm64`; for SWE-bench use its `--task-repo` local-build path. Because arm64 is experimental upstream, re-run the `FAIL_TO_PASS`/`PASS_TO_PASS` checker on arm64 before trusting the reward — an arch-specific test flake would silently corrupt the RLVR signal.
5. **Move SWE to the amd64 GPU/Instances substrate.** Instances run **amd64**, allow far larger images, and add a **persistent EBS volume** (so a one-time repo/deps install survives stop/resume) and **14-day** sessions (§"Two compute types"). This sidesteps both the arch and the image cap at once; the cost is spending an EC2 instance (and its GPU) that a CPU-only SWE checker does not otherwise need.

This is the concrete reason the §“Can AgentCore host every frontier sandbox?” table marks the per-repo Docker path with a ◑ rather than a clean ✓: for SWE, the sandbox *image* — not the checker or the GPU — is the binding constraint.

3.5 One image per Runtime, but a different env per row: how AgentCore reconciles it

SWE-bench raises a sharper version of the same problem: **each task instance ships its own Docker environment** (a specific interpreter version, system packages, and pinned dependencies checked out at one commit) — hundreds of distinct images across the benchmark. AgentCore’s binding constraint is that a **Runtime ARN is bound to exactly one container image**, chosen once at `CreateAgentRuntime` (`containerConfiguration.containerUri`, an ECR image — §“Building a customized Runtime”). An `invoke_agent_runtime` call passes only a **`runtimeSessionId` + payload — never an image [D8]**, so you **cannot swap the Docker environment per invocation**. Three ways reconcile “one image per ARN” with “a different env per row” — and the practical answer is usually a hybrid:

- **A — Move the per-row difference *out of the image, into per-session setup* (this project’s native model).** The sandbox is task-agnostic (`write_file` / exec / `read_file`) and all task specificity already lives in files the harness writes into *that episode’s session* at seed time. Extend that: on turn 0 the harness runs `git clone` + `git checkout <base_commit>`, installs the pinned deps, and drops in the hidden `run_checks.py` — so “a different image per row” becomes “a different **setup script** per row, run inside one generic image.” Each `runtimeSessionId` is an isolated microVM, so row *i*’s installed env never leaks into row *j*. *Cost:* per-episode setup latency + **network egress** (public or VPC endpoints, [D11]). *Limit:* works when the delta is *installable at runtime*; it struggles when a row needs a **different interpreter version**, **root apt** the base image can’t grant, or a heavy compiled toolchain — those belong in the image.
- **B — A *pool* of Runtimes, one image per ARN, routed by row.** Register **K Runtime ARNs, each with a different baked image**, and have the harness pick the ARN by the row’s repo/family instead of round-robin (the *capability-routed* dispatch the Terminal-Bench demo uses, §“The Terminal-Bench demo”). But hundreds of per-instance images cannot each get a Runtime (operationally

heavy, per-region limits), so group at **repo-family** granularity (one `django` image, one `sympy` image, …), **not** per-instance.

- **C — The practical hybrid (family base image + per-row delta).** Bake the *expensive, shared* layer per repo-family ARN (interpreter version, system libs, repo at a common base) → B; install the *cheap per-commit delta* (that instance's pip deps) at seed → A. This keeps each image small — under the microVM's ~2 GB arm64 cap above — while one ARN still covers many instances. If images are simply too large or too x86-bound, fall back to the amd64 **Instances** substrate (workaround #5).

Status: A–C above are a **design answer**. The project also built a fourth, simpler route and then removed it on 2026-09-30: one Runtime per SWE-bench instance on its Epoch AI arm64 image (§"History: removed demos (2026-09-30)").

4 Foundations of AgentCore Runtime

Amazon Bedrock AgentCore [D1] is a family of managed services for running and securing agents and tools — any framework, any model, fully serverless. This report uses one of them, **AgentCore Runtime**, as the execution **sandbox** for RL. This section defines its three core concepts, its two compute types, how AgentCore manages their lifecycle, and how you build and deploy a customized Runtime — cross-checking the official documentation [D1–D13, D17] against AWS's own SDK and sample code [D14–D16].

4.1 The microVM at a glance — what it is, what runs in it, what it supports, how you customize it

Before the precise definitions, a compact answer to the four questions a practitioner asks first. Each is expanded in the subsection named after it.

- **What is the microVM?** The default AgentCore Runtime substrate is a **Firecracker-class, hardware-virtualized microVM** — a lightweight VM with its **own guest kernel, CPU, memory, and filesystem**, one per session, that still boots **your OCI container image** inside it and is **sanitized on termination**. It gives VM-strength isolation for untrusted, model-generated code while keeping container packaging. It is **CPU-only** (no GPU [D5]), runs

an **ARM64** image serving `POST /invocations` + `GET /ping` on `0.0.0.0:8080`, and lasts up to **8 hours** with a 15-minute idle timeout. When a task needs a GPU, Runtime switches to the **Instances** (EC2) substrate instead (§"Two compute types"). *Details:* §"Runtime, session, and microVM" and §"microVM vs a Docker container vs E2B".

- **What environment / dependencies does it have?** Only what **your image** provides — there is no fixed "AgentCore OS" you inherit. A minimal sandbox is a slim Python base plus the `bedrock-agentcore` SDK; the built image must stay **ARM64 and under ~2 GB**. You add dependencies three ways, in increasing weight: bake them into the image; **install at `CMD`** when the container boots (how the GPU image adds the large CUDA torch wheel without blowing the 2 GB cap); or **install at seed time** into the per-session ephemeral workspace (`git clone`/`pip`/`apt` on turn 0, needing egress). Each session's installs are isolated and wiped on termination. *Details:* §"Language and environment per family" and §"Building a customized Runtime".
- **What tasks can it support?** Effectively every **CPU** sandbox family frontier labs train against: code + unit tests, math/SQL checkers, instruction following, tool-use simulators, RAG, and light CTF all fit one **generic** image (task specificity is just *files* seeded per session). **SWE-bench**-style per-repo images fit with workarounds (arm64 cross-build, seed-time clone, or moving to the amd64 Instances substrate). The one family the microVM **cannot** serve is **GPU kernels** (`nvcc`/`allclose`) — that needs the GPU Instances substrate. *Details:* §"Language and environment per family" and §"Can AgentCore host every frontier sandbox?".
- **How do you customize it as a Runtime?** Author a container that satisfies the contract (ARM64; `0.0.0.0:8080`; `/invocations` + `/ping`), cross-build it with `docker buildx --platform linux/arm64`, push to ECR, and register it with `CreateAgentRuntime`'s `containerConfiguration.containerUri`. AWS also offers a lower-effort **`agentcore` CLI** (CodeZip or Container build) and a **`bedrock-agentcore` Python SDK** whose `@app.entrypoint` implements the HTTP contract for you. *Details:* §"Building a customized Runtime".

4.2 Runtime, session, and microVM — three concepts, precisely

These three words are used loosely elsewhere; the integration depends on keeping them distinct.

- **Runtime (agent runtime).** A *versioned, containerized host* you create with `CreateAgentRuntime` [D9]. It packages your code — a container image in ECR, or a zipped Python bundle — behind an ARN. Creating a Runtime auto-creates version **V1** and a **DEFAULT endpoint**; you can publish further versions and custom endpoints and route invocations to them. The Runtime is the *deployable unit*; it holds no per-user state of its own.
- **Session.** One logical task or conversation, addressed by a **`runtimeSessionId`** you pass on `InvokeAgentRuntime` [D8] (client-supplied or Runtime-generated; **minimum 33 characters**, max 256, carried in the `X-Amzn-Bedrock-AgentCore-Runtime-Session-Id` header). The **first** invoke for a new id **cold-starts** dedicated compute; every **subsequent** invoke with the same id **re-routes to that same warm compute**, so files, memory, and processes persist *across turns within an episode*. Different ids are fully isolated.
- **microVM.** The *unit of isolation* backing a session on the default compute: a lightweight virtual machine with its **own CPU, memory, and filesystem**, sanitized on termination. One session maps to one microVM; a microVM is never shared across sessions. Its container contract [D17]: an **ARM64** image listening on **`0.0.0.0:8080`** that serves **`POST /invocations`** (the work) and **`GET /ping`** (health, returning `{"status": "Healthy"}` or `"HealthyBusy"`) — the same contract AWS's SDK scaffolds for you [D15].

The cardinality. These three nest **one-to-many-to-one**: **one Runtime serves many concurrent sessions, and each session maps to exactly one microVM (1 : N : 1)**. The Runtime is a stateless *template* (one image, one ARN); every distinct `runtimeSessionId` you invoke against it mints its own isolated microVM, and AgentCore's capacity provider places each one for you — so the session id, not a second Runtime, is how you get a second sandbox. You create *additional* Runtimes only when the **image**, the per-region **session cap**, or the **AZ/account** must differ (§"Scaling the loop"); otherwise a single Runtime backs all your concurrent sessions. Put another way: **the number of Runtimes tracks the number of distinct sandbox *images* (capabilities), not the number of tasks** — because the sandbox is task-agnostic (`write_file` / `exec` / `read_file`), one generic image serves every task family that runs on the same substrate, so even a mixed-family training run stays on **one Runtime, many sessions**.

In RL terms: **the Runtime is the deployable sandbox image, the session is one episode, and the microVM is the isolated box that episode executes in.** So N concurrent episodes = N session ids on one Runtime — not N Runtimes.

4.2.1 A mixed-family batch — how many Runtimes, sessions, and microVMs, and who picks the image?

The sharpest way to feel the distinction is a **heterogeneous rollout step**: a mixed-family dataset (the removed `demo_cpu_tasks` with a comma-list `DATASET=`, e.g. `math,sql,ctf,tool_use`) draws a batch whose rows span several families — say a math problem, a text-to-SQL query, a CTF challenge, and a tool-use task — with `n-samples-per-prompt` episodes each. A natural worry is that four task types need four sandbox images, so four Runtimes. They do not. Count each concept:

- **Runtimes: one** (a small pool of *K* only for throughput, not for variety). Every one of these families is a **pure-CPU / stdlib checker** — `sympy`, `unittest`, `sqlite3`, an isolated shell, tool stubs — so a **single generic CPU image** runs and checks all of them. The Runtime count tracks distinct *images* (capabilities), and here there is exactly one capability. You would add a second Runtime only for a genuinely different image — the **GPU-kernel** family (`nvcc`, a different substrate) — not for a different CPU family.
- **Sessions: one per in-flight episode, recycled.** If the batch is `RB` prompts × `NS` samples, that is `RB×NS` *episodes*, but they do **not** all run at once: only `S = CONC × num_engines` are concurrent, and those few sessions **stream through all `RB×NS` episodes** over the step. A math episode and a SQL episode are just two different session ids against the same Runtime.
- **microVMs: one per session, 1:1.** Each `runtimeSessionId` mints its own isolated microVM (booting that same generic image), sanitized when the session ends. So at any instant there are `S` live microVMs, whatever mix of families they happen to be running.

So how do slime and AgentCore know which image a given data row needs? They don't — and by design, they don't have to. No per-row image selection happens anywhere, because the image is *task-agnostic*. The routing that *does* happen is a division of labor:

- **slime / the harness picks the *task*, not the image.** Each dataset row carries `metadata.task.family`; the generate hook (`_task_from_sample`) rebuilds the matching **Task subclass** — its prompt, seed files, and hidden checker — and the harness **writes those task-specific files into that episode's session** at seed time (`write_file`). The family discriminator chooses *what gets written into the box*, never *which box*.

- **AgentCore never sees a task at all.** An `invoke_agent_runtime` call carries only a **`runtimeSessionId` + payload — never an image** [D8]; the image was fixed once, at `CreateAgentRuntime`. AgentCore's only job is to route the session id to its microVM. It is oblivious to whether the payload is a math check or a SQL query.

In other words, "a different environment per row" is realized as **a different set of *files* per session inside one shared image**, not a different image per row. The one time a row truly needs a different *image* — a CPU family vs. the GPU-kernel family — that is a **substrate** choice made per run by an env var, not per row by a router; genuine per-row *image* routing is what the Terminal-Bench demo does, one Runtime per task (§"The Terminal-Bench demo"). This is exactly why a mixed-family run stays on **one Runtime, S sessions, S microVMs**.

4.3 microVM vs a Docker container vs E2B

Because "sandbox" and "container" are often used interchangeably, it is worth being precise about what the microVM actually is — especially since we are running **untrusted, model-generated code**, where the strength of the isolation boundary is the whole point.

- **A Docker container** isolates via Linux namespaces and cgroups but **shares the host kernel**. That is fine for packaging and for trusted workloads, but a shared kernel is a comparatively soft boundary for hostile code: a kernel exploit escapes to the host and to co-tenants. Containers alone are not a safe multi-tenant boundary for arbitrary generated code.
- **An AgentCore microVM** is a lightweight, **hardware-virtualized** virtual machine (a Firecracker-class microVM) with its **own guest kernel**, its own CPU/memory/filesystem, and a VM-strength boundary — while still booting **your OCI container image** inside it. So it is not "container *or* VM": you keep container packaging *and* gain VM isolation, provisioned per session and sanitized on termination. On top of that it is **AWS-native** — reached only by in-account SigV4/IAM, placeable in your VPC, and integrated with the other AgentCore services (Gateway, Memory, Identity, Observability) — and it can carry a **GPU** (via Instances) and **long sessions** (8 h microVM / 14 d Instances).
- **E2B** is an independent sandbox platform that, like AgentCore, runs code in **Firecracker microVMs** (its templates are also defined from Dockerfiles), so

the *isolation model is comparable*. The differences are operational, not architectural: E2B is its own cloud (or self-hosted) service with its own SDK and identity, oriented to short-lived code-interpreter sandboxes, whereas AgentCore is a managed AWS service that runs **in your own account** under IAM/VPC, adds **GPU on Instances** and **multi-hour/multi-day sessions**, and slots into the rest of the Bedrock AgentCore stack.

	Docker container	AgentCore microVM	E2B sandbox
Isolation boundary	shared host kernel (namespaces/cgroups)	**own kernel** — Firecracker-class microVM	own kernel — Firecracker microVM
Runs your image?	yes (OCI)	yes — your OCI image *inside* a microVM	Dockerfile-defined templates
Managed by	you / your orchestrator	**AWS, in your account** (IAM/SigV4/VPC)	E2B service (cloud or self-host)
GPU	host-dependent	**via Instances** (EC2)	not the core offering
Session length	you manage	8 h microVM / 14 d Instances	short-lived (code-interpreter)

For an RL sandbox running untrusted policy output, the microVM's per-session hardware isolation is the property that matters: one episode's generated code cannot see another episode's files, the trainer's weights, or the host — and the box is destroyed and its memory wiped when the session ends.

4.4 Two compute types — managed serverless (microVM) vs Instances (GPU EC2)

AgentCore Runtime offers two substrates, chosen at create time; the loop code is identical across them.

- **Serverless microVM (default).** No servers to manage: AgentCore provisions a per-session microVM on demand. **CPU only** — a microVM has *no GPU* [D5]. Sessions last up to **8 hours** with a **15-minute idle timeout** (reset on each invoke). This fits the large majority of RL sandboxes: code execution + unit tests, math/SQL checkers, tool simulators.
- **Instances (GPU).** When a task needs a GPU, Runtime runs on the **Instances** compute type — real **EC2 instances in your own account**, provisioned through a **capacity provider** (`capacityProviderConfiguration` → `ec2Configuration`) [D5, D9]. Supported GPU families include **g4dn, g5, g6, g6e, gr6, g6f, gr6f, g7e** (plus `inf2`); AgentCore provisions the NVIDIA drivers, so a standard CUDA image works unmodified. Sessions can run up to **14 days**, a **persistent EBS volume** survives a session stop and re-attaches on resume, and — unlike the microVM — one instance/session can host **many agents** (1:N). Crucially, one `runtimeSessionId` still maps to exactly **one instance = one GPU**, so there is **no cross-session GPU time-slicing**: you scale GPUs by adding sessions/instances, not by fractioning one.

> **What about a multi-GPU instance — two sessions, one GPU each?** No, not natively: **one AgentCore session = one whole instance** (session→instance is 1:1; the capacity provider hands you the *instance*, not a GPU). So you **cannot** point two distinct `runtimeSessionIds` at the *same* 2-GPU box and expect one GPU each — two session ids try to acquire two *separate* instances, and you never get to pin a session to a particular GPU on a shared box. To put several episodes on the GPUs of *one* multi-GPU instance you must instead use **one shared session** plus a **self-managed in-instance farm** (a Ray farm keyed by per-episode `sub_session`), which is what the farm demo `demo_gpu_farm` did until its removal on 2026-09-30 — `B4_GPU_FRACTION=1.0` with `B4_FARM_GPUS=N` leased **one whole GPU per sub-session** (`FARM=whole`), and `B4_GPU_FRACTION=0.5` on a single GPU packed two fractional episodes onto it (`FARM=fractional`) — no MIG and no cross-session time-slicing required, but also no VM boundary between episodes. The shipped demos use no Instance at all (the removed `demo_gpu_single` ran one session per g6).

> **Can you pick a CPU-only EC2 instance for an Instance?** Not as a documented path. The `ec2Configuration` families AWS lists [D5, D9] are all **accelerated** (g4dn/g5/g6/g6e/gr6/g6f/gr6f/g7e + `inf2`) — there is **no general-purpose (m/c/t) CPU family**. The split is deliberate: **CPU-only work be-**

> **longs on the serverless microVM**, and the Instances substrate exists to add a **GPU**. So if you want a larger/x86/longer-lived *CPU* box (e.g. to escape the microVM's arm64 + 2 GB image limits, §3), the documented route is to launch a GPU-family Instance and leave the GPU idle — you pay for an accelerator you don't use. (The supported list is per the docs as of this work; re-check `ec2Configuration`. In practice this project's account accepts an `m5.xlarge` capacity provider: the removed `demo_cpu_instance` ran one m5 Instance per session on `acs_cpu_cp_instance`.)

Property	Serverless microVM	Instances (GPU)
Compute	managed, per session	EC2 in your account (capacity provider)
GPU	none [D5]	g4dn/g5/g6/g6e/g7e… + inf2
vCPU / RAM	**allocated per session; not a customer-set knob** — metered by vCPU-hour + GB-hour [D2, D13]	the chosen EC2 type's spec (e.g. `g6.xlarge` = 4 vCPU / 16 GB RAM / 1× L4)
Disk	**ephemeral only — no persistent volume**; in-VM state lost at session end [D3]	ephemeral **plus a sized, persistent EBS volume** (survives stop/resume)
Container image	**arm64**, built image **≤ ~2 GB** (project-measured, §"Building a customized Runtime")	**amd64**, far larger images allowed
Max session	8 h	14 d
Isolation unit	1 microVM / session	1 instance / session (1 GPU)
Filesystem	ephemeral	persistent EBS across stop/resume
Agents / session	1	many (1:N)

On the microVM's resource envelope. Two limits bite in practice and are worth stating plainly. First, **the filesystem is ephemeral**: "any data stored in memory or written to disk persists only for the compute lifecycle" [D3], so there is *no* persistent disk on the serverless substrate — a repo you `git clone` or a wheel you install lives only for that session (up to 8 h) and is wiped on termination; durable state needs **session storage / Memory**, or the Instances substrate's EBS volume. Second, the **built container image must stay small** — this project measured a **~2 GB** ceiling (§"Building a customized Runtime"), which is why the GPU image installs its multi-GB CUDA wheel at `CMD` rather than baking it in. AWS does **not** publish a fixed per-session vCPU/RAM *number* for the microVM — capacity is provisioned per session and billed by **vCPU-hour and GB-hour** (the `agent.runtime.vcpu.hours.used` / `agent.runtime.memory.gb_hours.used` usage metrics [D13]), so treat CPU/RAM as *metered, not sized*, and confirm the current hard caps on the **Service Quotas** page for your account before sizing a heavy checker. When a task genuinely needs a large, fixed RAM/disk footprint (a big SWE repo image, a data-heavy checker), that is a reason to move it to **Instances**, where the resources are simply the EC2 instance type's and the EBS volume you attach (§3, §"Two compute types").

4.5 How AgentCore manages the lifecycle — Runtime, session, microVM, and GPU instance, end to end

An RL workload touches **four** AgentCore objects, with **nested lifetimes**. Keeping them straight is what makes cost and capacity predictable; conflating them is what makes a run's instance count surprising. The whole section below is one answer to a single question — *from the moment you deploy to the moment the run ends, what is created, how long does it live, and what ends it?*

Object	Created by	Lives for	Ended by	How many in a run
Runtime (the deployable + endpoint)	`CreateAgentRuntime`, at **deploy time**	the **whole run** (and beyond)	`DeleteAgentRuntime`	**1** by default; a small pool only to raise invoke throughput

Object	Created by	Lives for	Ended by	How many in a run
Session (`runtimeSessionId`)	first invoke with a **fresh id**	**one episode**	`StopRuntimeSession` / idle (15 min) / max-life	≈ **CONC** live at once, **recycled** through every row
microVM (serverless / CPU substrate)	provisioned on session launch	its **session**	session end → **memory sanitized**	1 per live CPU session
GPU instance (EC2, on Instances)	**cold-launched** on session launch	its session (the instance may stay **warm** in the capacity provider)	session end / capacity-provider scale-in	1 instance = **one GPU**, ≈ CONC live

The Runtime is a deploy-time artifact — it spans the whole run. `CreateAgentRuntime` builds version V1 and the DEFAULT endpoint; updates publish new versions; you route traffic by endpoint. Deleting the Runtime removes the deployable — **sessions are not part of the Runtime's own state.** slime is simply handed the ARN and never creates or deletes it (§"Who controls the AgentCore lifecycle"). A run normally has **one** Runtime; pooling **K** of them raises invoke throughput but changes neither how many tasks nor how many images you can serve (§"A mixed-family batch").

The session is the RL unit — one per episode, Active → Idle → Terminated [D3, D4]. The generate hook mints a fresh `runtimeSessionId` per episode — *"one session id == one isolated sandbox for the whole episode"* — so the session id is the isolation unit.

- **Launch / cold start.** There is no "create session" API. The first `invoke_agent_runtime` under a fresh id (the hook's `__aenter__` sends a `ping`) provisions the compute: a microVM on the serverless substrate, or a **cold-launched EC2 instance** on Instances (*minutes* for a GPU box). Same-id invokes thereafter hit warm compute.

- **Active / Idle.** Each op (`write_file` / `exec` / `read_file`) is one more invoke under the *same* id and resets the **15-minute idle timer**; between invokes the session sits Idle but warm.
- **Termination.** Explicit via **`StopRuntimeSession`** [D12] (this project now calls it at episode end — see below), or automatic on **idle expiry (15 min)**, **max lifetime** (microVM **8 h** / Instances **14 d**), or a failed health check → the microVM is **terminated and its memory sanitized**.

The microVM and the GPU instance are the compute a session rides on. One live session == one microVM (CPU) or **one EC2 instance = one GPU** (Instances), a strict 1:1 with **no cross-session GPU time-slicing** (§"Two compute types"). The microVM is **ephemeral** — its filesystem and memory vanish at session end. A GPU instance's **EBS volume persists across a stop**, and the capacity provider may keep the *instance* warm for the next session even after the *session* ends. So the session's lifetime and the instance's lifetime are related but not identical: the session is what your episode owns; the instance is what the fleet manages.

Terminate at episode end — this project now stops sessions explicitly (per backend). A session can be ended explicitly or abandoned to the idle timer. The three sandbox backends differ at episode end (`__aexit__`):

Backend (`SANDBOX=`)	At episode end	Session ends by
`agentcore` (Code Interpreter microVM)	calls the client `stop()`	**explicit**, immediately
`agentcore-runtime` (sandbox-as-Runtime)	calls **`StopRuntimeSession`** (best-effort; `AGENTCORE_STOP_SESSION=0` to opt out)	**explicit**, immediately — idle reclaim is only the fallback if the stop call fails
`agentcore-ray-farm` (B4; no shipped demo since 2026-09-30)	sends `op=close` for the sub_session	frees the **fractional-GPU actor** immediately; the shared g6 session persists

Because each episode mints a **fresh** `runtimeSessionId`, there is no warm session to preserve across episodes: stopping at `__aexit__` returns the microVM (and its concurrent-session quota slot) **immediately** instead of holding it idle for up to 15

minutes — which matters when a batch runs many episodes concurrently against a bounded session quota. Set `AGENTCORE_STOP_SESSION=0` for a GPU fleet that deliberately keeps instances warm for the next session, letting idle reclaim apply instead.

A batch of prompts — row count ≠ instance count. A 100-row dataset (KernelBench level_1) does **not** launch 100 instances. Rows stream through episodes across training steps; at any instant only `--sglang-server-concurrency` (default **4**) episodes are in flight, so the live-instance count = concurrent sessions ≈ CONC, and those few instances **recycle through all rows** over time. What multiplies instances is *concurrency*, not data:

- **One session per episode (`agentcore-runtime` on an Instance, e.g. a g6 or an m5):** one session per episode × 1:1 ⇒ **N concurrent episodes = N concurrent instances** (bounded by `HARNESS_WORKERS` + `PREOPEN` and the capacity provider's max-instance cap).
- **Historical — B4 (`agentcore-ray-farm`, `demo_gpu_farm`, removed 2026-09-30):** all concurrent episodes shared **one** `runtimeSessionId` → **one** g6 instance; each episode's sub_session leased a fractional-GPU Ray actor, so N concurrent episodes still = one instance.

This is the concrete cost lever: packing episodes onto one instance (B4) raises sandbox concurrency on a GPU task **without** paying for N instances, but gives up the VM boundary between episodes; the project now pays for the instances (one session per episode) and hides the cold boot with pre-open.

A mixed-task batch — same session budget, different task per row. When a batch spans task families (a comma-list `DATASET=`), the lifecycle above is **unchanged**: one Runtime image serves every family, and the harness writes each row's task files into its session and picks the *task*, not the image (§"A mixed-family batch"). A mixed-family batch therefore consumes exactly the same Runtime / session / microVM / instance budget as a single-family batch of the same size — the family mix changes what runs *inside* each session, not how many sessions exist. Capability routing (a different image per family) changes that by one Runtime per extra image — the Terminal-Bench demo does exactly this at task scale (one Runtime per task, strict routing).

Latency — measure, don't assume. The docs promise "fast cold starts" but publish no number [D2]; a GPU first-invoke can take **minutes** to launch

the EC2. This project therefore instruments every sandbox round-trip ([sbx-op] → perf/sandbox_ms_{p50,p95}) and every generation ([sglang-gen] → perf/sglang_gen_ms_*) — the same signals the scaling analysis in §"Scaling the loop" depends on.

4.6 Building a customized Runtime — bring your own container, end to end

AWS offers three ways to put code on a Runtime — the agentcore CLI's zero-Docker **CodeZip** build [D16], the **bedrock-agentcore** Python SDK's @app.entrypoint [D15], and **bring your own container**. This project uses the **bring-your-own-container** path, because it is the only one that gives you the whole image: base OS, system packages, CUDA toolchain, a cloned repo — exactly what a task-specific sandbox needs. CreateAgentRuntime takes a **union** — containerConfiguration (an ECR image) **or** codeConfiguration (a zip); BYO container is the former [D9]. This section walks the full path from a Dockerfile to a training run that uses it.

Step 1 — write a container that satisfies the contract. AgentCore only requires that your image serve HTTP on 0.0.0.0:8080 with two routes — **/invocations** (one request = one op) and **/ping** (health) [D17] — and, for the managed microVM fleet, that it be built for **arm64**. The bedrock-agentcore SDK's BedrockAgentCoreApp + @app.entrypoint implement both routes, so the image stays tiny. A minimal generic sandbox image (build context: the demo folder, which vendors agentcore_slime_demo/):

```
FROM public.ecr.aws/docker/library/python:3.12-slim
WORKDIR /app
RUN pip install --no-cache-dir bedrock-agentcore
# sandbox_server_agent.py is self-contained (stdlib + bedrock_agentcore, no relative
# imports), so copy it flat — no package, nothing heavy.
COPY agentcore_slime_demo/sandbox_server_agent.py /app/sandbox_server_agent.py
ENV SANDBOX_WORKDIR=/workspace
EXPOSE 8080
CMD ["python", "-m", "sandbox_server_agent"]      # BedrockAgentCoreApp serves :8080
```

The Terminal-Bench demo does not write its own image: its deploy script (deploy_terminalbench.py) appends one wrapper layer to each task's own Dockerfile, so the task's interpreter and files are untouched:

```
USER root
COPY py/ /opt/acs/py/                          # standalone arm64 CPython
COPY acs_http_server.py sandbox_server_agent.py /opt/acs/
ENTRYPOINT []
CMD ["/opt/acs/py/bin/python3", "/opt/acs/acs_http_server.py"]   # stdlib server on
↪  :8080
```

A **GPU** sandbox (the removed GPU demos' image) changes two things: AgentCore injects `libcuda.so.1` (point LD_LIBRARY_PATH at it), and the CUDA torch wheel is large — so install it **at `CMD`, not baked into the image**, to stay under AgentCore's **~2 GB** image cap (§"Two compute types"):

```
ENV LD_LIBRARY_PATH=/usr/lib64          # AgentCore injects libcuda.so.1 here
CMD ["sh","-c","pip install torch==2.5.1 --index-url
↪  https://download.pytorch.org/whl/cu124 \
               && exec python -m sandbox_server_agent"]
```

Step 2 — create the ECR repo and log Docker in.

```
export AWS_REGION=us-west-2
ACCT=$(aws sts get-caller-identity --query Account --output text)
REG=$ACCT.dkr.ecr.$AWS_REGION.amazonaws.com
aws ecr create-repository --repository-name acs-sandbox-cpu --region $AWS_REGION  #
↪  idempotent
aws ecr get-login-password --region $AWS_REGION | docker login --username AWS
↪  --password-stdin $REG
```

Step 3 — cross-build for arm64 and push straight to ECR. The managed microVM fleet is **arm64** (`CreateAgentRuntime` rejects amd64 with *"Supported platforms: [arm64]"*), while a GPU capacity provider is **amd64** — so a build host of the other arch must cross-build. Use a `docker-container` buildx builder (with QEMU) and emit a **plain** single-arch image — `--provenance=false --sbom=false` keeps it out of the attestation-manifest-list shape that `containerUri` rejects:

```
docker buildx create --name xbuild --driver docker-container --bootstrap   # once
docker buildx build --builder xbuild --platform linux/arm64 \
  --provenance=false --sbom=false \
  -f Dockerfile.sandbox -t $REG/acs-sandbox-cpu:latest --push .
```

Step 4 — register the image as a Runtime. An **execution role** (trusts `bedrock-agentcore.amazonaws.com`; grants ECR pull + CloudWatch logs; for a harness Runtime that calls the sandbox, also `InvokeAgentRuntime`) is passed to `create_agent_runtime`, whose `agentRuntimeArtifact` carries the `containerUri`:

```
control = boto3.client("bedrock-agentcore-control", region_name=REGION)
resp = control.create_agent_runtime(
    agentRuntimeName="acs-sandbox-cpu",
    roleArn=exec_role_arn,
    agentRuntimeArtifact={"containerConfiguration": {"containerUri":
↪   f"{REG}/acs-sandbox-cpu:latest"}},
    protocolConfiguration={"serverProtocol": "HTTP"},
    networkConfiguration={"networkMode": "PUBLIC"},   # or VPC + networkModeConfig
    environmentVariables={"SANDBOX_WORKDIR": "/workspace"},
)
# poll DescribeAgentRuntime until status == READY, then keep the ARN:
sandbox_arn = resp["agentRuntimeArn"]
```

For a **GPU** sandbox this step is preceded by **`CreateCapacityProvider`** (`ec2Configuration`: instance type e.g. `g6.xlarge`, VPC subnets/SGs, EBS, `idleInstanceTimeout=600`, `maxLifetime=86400`), attached via `capacityProviderConfiguration` [D5, D9]; the execution setup then also needs the operator policy `BedrockAgentCoreRuntimeInstancesOperatorRolePolicy`.

The Terminal-Bench demo bundles these steps into its numbered scripts. Both are idempotent and resumable: `bash 2_build_image.sh` does the ECR create, buildx and push per task, and `bash 4_deploy_runtime.sh` calls `terminal-bench/deploy_terminalbench.py`, which uses `deploy/agentcore_deploy.py` to mint the roles, register one VPC Runtime per image and write the image → ARN map:

```
cd code/demos/demo_terminalbench_kernelbench
bash 2_build_image.sh      # ECR create → buildx --platform linux/arm64 → push, per
↪   task
bash 4_deploy_runtime.sh   # exec role → CreateAgentRuntime (VPC) per image →
↪   arn_by_image.json
```

Step 5 — use it in a training run. Nothing in slime references the image or the ARN at build time; the sandbox client reads the ARN from the environment at rollout time (and raises if unset). So "using" a custom Runtime in training is one export — the ARN that Step 4 printed — before launching `run.sh` (a pool like Terminal-Bench's exports the map `AGENTCORE_SANDBOX_RUNTIME_ARN_BY_IMAGE` instead):

```
export AGENTCORE_SANDBOX_RUNTIME_ARN=arn:aws:bedrock-agentcore:us-west-
↪   2:<acct>:runtime/acs-sandbox-cpu-⋯
↪
export AGENTCORE_REGION=us-west-2
```

```
export SANDBOX=agentcore-runtime          # select the sandbox-as-Runtime backend
bash run.sh                               # slime's generate hook now invokes YOUR
↪  Runtime per op
```

From here the lifecycle in §“How AgentCore manages the lifecycle” takes over: the first `invoke_agent_runtime` under each episode’s fresh session id cold-starts a microVM (or GPU instance) from **your** image, every tool call is one more invoke, and `__aexit__` stops the session at episode end.

5 Using AgentCore as the sandbox in an agentic RL workflow

With the foundations in place, this section shows how AgentCore Runtime plugs into a real RL trainer — **slime** — as the execution **sandbox**, with the harness running **inside the sandbox session**. The integration is deliberately small: **no fork of slime**, two hook flags, and a sandbox client that turns each tool call into one `InvokeAgentRuntime`.

5.1 Where AgentCore attaches to slime — hooks, not a fork

slime [9] runs Megatron (training) and SGLang (rollout) asynchronously (§“slime — the trainer”). It exposes the rollout path as a **custom generation function**, so an agentic workload plugs in without touching the training kernel:

- `--custom-generate-function-path agentcore_slime_demo.generate.generate` — the per-prompt rollout: it hands the episode to the harness tier (`harness_pool.py`), which takes an AgentCore sandbox session, then an SGLang engine, and runs the mini_claude loop inside that session.
- `--custom-rm-path` — an optional custom reward hook (here the reward comes straight from the in-sandbox checker, so the generate hook attaches it directly).

The key design choice: **AgentCore is the sandbox, and the harness rides in it.** The harness (agent loop) is independent per episode, so it runs inside the episode’s own sandbox session and calls the trainer’s per-engine `AnthropicAdapter` back over the VPC; the trainer keeps only the dispatch tier and the adapters, which record the exact tokens. There is no in-trainer fallback: a session that cannot host the loop

(no `python3` in the image, or no route back to the adapter) fails the episode with `HarnessPlacementError`. The legacy two-runtime path (a separate harness Runtime, `agentcore_runtime_generate.py` / `harness_runtime_agent.py`) was deleted.

5.2 The call path — one diagram

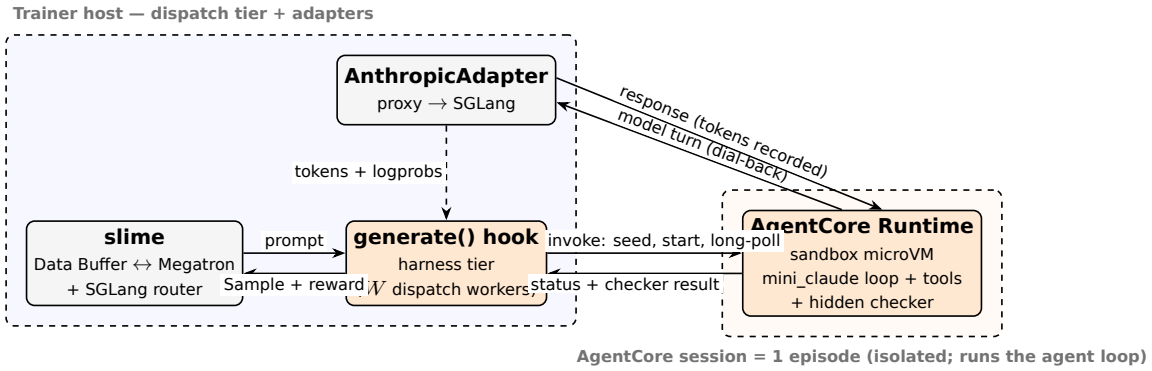


Figure 7: How AgentCore drops into slime. slime hands a **prompt** to the `generate()` hook, whose **harness tier** queues the episode: a free dispatch worker takes a fresh **AgentCore sandbox** session, then the least-loaded SGLang engine, seeds the task, and starts the **mini_claude agent loop inside the session**. Each **model turn** dials back from the session to that engine's **proxy** (`AnthropicAdapter`), which records the exact sampled **tokens + logprobs** — the TITO property of §"Token-in-token-out"; each **tool call** runs locally in the session. The trainer only long-polls the loop over `invoke_agent_runtime`, then runs the hidden **checker** (the reward source). The adapter's record becomes the loss-masked **Sample + reward** in slime's buffer. The trainer host keeps only the dispatch tier and the adapters; the loop and its tools live in the isolated per-episode microVM.

5.3 Key code — three pieces

1. The generate hook (the harness tier). slime calls this once per prompt. It queues the episode on the harness tier: a free dispatch worker takes **one fresh sandbox session**, then the SGLang engine with the fewest in-flight episodes (sticky for all its turns, §"Scaling the loop"), runs the mini_claude loop inside that session, and returns a single loss-masked `Sample`:

```python
# agentcore_slime_demo/generate.py  (wired via --custom-generate-function-path)
async def generate(args, sample, sampling_params, evaluation=False):
    task = _task_from_sample(sample)                        # rebuild Task from row
↪  metadata
    kind, image = os.environ.get("SANDBOX", "agentcore-runtime"), task.image
    pool = harness_pool.get_pool(args)                      # the harness tier (W
↪  workers)

    async def episode(worker):                              # run by a FREE worker
    ↪  (pull queue)
        async with pool.sandbox((kind, image), lambda: maybe_keep_warm(
                make_sandbox(kind, image=image))) as (sb, wait_ms, slot):  # sandbox
↪  FIRST
            with pool.engine() as (e, engine):              # then the least-loaded
            ↪  engine
              # mini_claude runs INSIDE sb and dials this engine's adapter back; the
               # adapter's token record becomes ONE loss-masked Sample + the reward
               return await mini_claude_episode.run(args, sample, task,
                ↪  sampling_params,
                                              sb, engine, fields=pool.tier_fields(
                                                      worker, e, wait_ms, slot))

    return await pool.run(episode)                          # queue it; await the bare
    ↪  Sample
```

Two RL-critical details: the loss mask is **1 on the policy's own tokens, 0 on prompt/observation/tool tokens**, and the adapter's per-session record (`finish_session`) reuses the **recorded `token_ids` per turn** — no re-tokenization, the TITO property of §"Token-in-token-out". The pool returns a *bare* `Sample` (not a list): slime's fully-async path (the only rollout driver) chokes on nested sample lists.

2. The sandbox client (one op = one invoke). Each `write_file` / exec / `read_file` becomes a single SigV4 `invoke_agent_runtime`, keyed by the episode's session id. boto3 is synchronous, so it is offloaded with `asyncio.to_thread`, letting N episodes run concurrently (the harness tier sizes the loop's default executor to the pool, so the thread pool is not a hidden cap):

```python
# agentcore_slime_demo/agentcore_runtime_sandbox.py
def _invoke_sync(self, payload: dict) -> dict:
    resp = self._client.invoke_agent_runtime(               # boto3
↪  "bedrock-agentcore"
        agentRuntimeArn=self._arn,
        runtimeSessionId=self._session_id,                  # >=33 chars; one per
↪  episode
        payload=json.dumps(payload).encode())
```

```
    return json.loads(resp["response"].read().decode())

async def exec(self, cmd, *, timeout=120, **kw):
    res = await asyncio.to_thread(self._invoke_sync,
              {"op": "exec", "command": cmd, "timeout": int(timeout), "workdir":
↪  self._workdir})
    return int(res.get("exit_code", 1)), res.get("stdout", ""), res.get("stderr", "")
```

The real client also **retries** transient 5xx/throttle blips with backoff (one bad round-trip must not crash a multi-hour run) and carries a much larger budget for GPU "insufficient capacity" while a g6 launches.

3. The Runtime server (what runs inside the sandbox). On the Runtime, the `bedrock-agentcore` SDK's `@app.entrypoint` receives each payload and dispatches on `op`. This is the whole sandbox server — and *the checker runs here, so the microVM is the reward source*:

```
# agentcore_slime_demo/sandbox_server_agent.py  (deployed as the Runtime image)
from bedrock_agentcore.runtime import BedrockAgentCoreApp

def handle(request: dict) -> dict:              # pure dispatch (unit-testable
↪  without AgentCore)
    op = (request or {}).get("op", "ping")
    if op == "ping":       _ensure_workdir(request.get("workdir")); return {"ok":
↪  True}
    if op == "write_file": return _do_write(request)     # -> {"ok": True}
    if op == "read_file":  return _do_read(request)      # -> {"text": ...}
    if op == "exec":       return _do_exec(request)      # -> {exit_code, stdout,
↪  stderr}  (runs the checker)

app = BedrockAgentCoreApp()

@app.entrypoint
def entrypoint(request):
    return handle(request)

app.run()      # serves 0.0.0.0:8080 — POST /invocations + GET /ping
```

5.4 Who controls the AgentCore lifecycle — deploy driver vs. the hook

A precise point that the code above implies but is worth stating outright: **slime's training kernel controls neither the Runtime nor the session.** Lifecycle is

split across two planes, and slime's core (`RolloutManager`, Data Buffer, Megatron, the SGLang router) is entirely **AgentCore-unaware** — it only calls the function registered at `--custom-generate-function-path` and gets a `Sample` back.

- **Runtime lifecycle (create/delete the ARN) — outside slime, at deploy time.** No slime component calls `CreateAgentRuntime`/`DeleteAgentRuntime`. The Runtime (and, for GPU, the capacity provider) is provisioned **once before training** by the deploy driver (`terminalbench/deploy_terminalbench.py`), and slime is simply handed the ARN via `AGENTCORE_SANDBOX_RUNTIME_ARN`, or a per-image map via `AGENTCORE_SANDBOX_RUNTIME_ARN_BY_IMAGE`. The sandbox client only *reads* that env var and raises if it is unset — it never creates the Runtime. The Runtime is a **deploy-time artifact**, not a training-loop object.
- **Session lifecycle (launch/terminate per episode) — the custom generate hook and its sandbox client.** All session control lives *below* slime's seam, inside `agentcore_slime_demo.generate.generate`, which opens one sandbox per prompt (`async with make_sandbox(...) as sb:`). The client (`AgentCoreRuntimeSandbox`) mints **one `runtimeSessionId` per episode**, and:
  - **Launch is implicit** — there is no "create session" API. `__aenter__` sends the **first `invoke_agent_runtime`** (a `ping`) under the fresh session id, and that first invoke is what **cold-starts the microVM** (and, on GPU, warms the g6).
  - **Each op** (`write_file` / exec / `read_file`) is one more `invoke_agent_runtime` under the *same* session id.
  - **Termination is explicit** — `__aexit__` calls **`StopRuntimeSession`** (best-effort; `AGENTCORE_STOP_SESSION=0` opts out) so the microVM and its concurrent-session quota slot are released **immediately** at episode end. AgentCore's **~15-minute idle timeout** remains the fallback if the stop call fails. slime's kernel still never terminates a session itself — the control lives in the hook's sandbox client.

Action	API	Controlled by	When
Create / delete Runtime (+ capacity provider)	`CreateAgentRuntime` / `Delete…` / `CreateCapacityProvider`	**deploy driver** — *not slime*	once, before/after the run

Action	API	Controlled by	When
Drive rollout	—	slime `RolloutManager` / Data Buffer — **AgentCore-unaware**	per prompt
Launch session (cold-start microVM)	first `invoke_agent_runtime` (`ping`)	the hook's sandbox client `__aenter__`	per episode
Run ops	`invoke_agent_runtime` (write/exec/read)	sandbox client, same session id	each turn
Terminate session	**`StopRuntimeSession`** (best-effort; idle timeout is the fallback)	the hook's sandbox client `__aexit__`	per episode, at end
Defer termination across weight-sync (optional)	repeated `ping`	`SessionKeeper`, *outside* the harness	during the sync barrier

Explicit `StopRuntimeSession` at episode end is the happy-path teardown, but a session can still be lost **mid-episode**: a long weight-sync pause between two turns can trip the 15-minute idle clock while the episode is still running and reclaim the session out from under it — exactly the failure mode the `HEARTBEAT=1` `SessionKeeper` heartbeat addresses (§"Hiding the container launch: pre-open and heartbeat"). **One-sentence version:** the deploy driver owns the Runtime, the `generate` hook + its sandbox client own the session (launch on first invoke, terminate with `StopRuntimeSession` at episode end), and slime's training kernel owns neither.

5.5 Ending the session at episode end — explicit teardown, and what "end" means per substrate

We always end the session explicitly when the episode ends — and *what* gets ended differs by substrate, deliberately. The teardown is not inside the loop: the episode (`mini_claude_episode.run`, and the in-session

loop it starts) runs *inside an already-open sandbox* and closes nothing itself — it grades and returns the sample. The close fires because **every caller wraps the episode in a context manager** — the harness tier's `async with pool.sandbox(...)` around `make_sandbox(...)` (`harness_pool.py`, `generate.py`) — so the sandbox's `__aexit__` runs the instant the episode returns. (With the opt-in `SANDBOX_SESSIONS=M`, a pooled session instead has its `ep-<hex>` workdir removed and goes back to the free list; the pool stops every pooled session at exit.) One teardown path, exercised identically by all demos.

What `__aexit__` does depends on the backend the `SANDBOX` env selected:

- **Per-episode session (managed microVM or capacity-provider Instance, `agentcore-runtime`; every shipped demo): stop the whole session.** `AgentCoreRuntimeSandbox.__aexit__` calls **`StopRuntimeSession(runtimeSessionId, agentRuntimeArn)`** (`agentcore_runtime_sandbox.py:287-324`; opt-out `AGENTCORE_STOP_SESSION=0`). Each episode minted its own `sbx-<uuid>` session, so ending the whole thing is exactly right — there is nothing to preserve.
- **Warm-shared instance (CPU/GPU Instances, `agentcore-ray-farm`; historical, its last demo `demo_gpu_farm` was removed 2026-09-30): close only the episode's `sub_session`, never the shared session.** `RayFarmRuntimeSandbox.__aexit__` sends a single **`op=close` for the per-episode `sub_session`** and releases its client-side concurrency slot (`agentcore_runtime_sandbox.py:484-493`); it **never** calls `StopRuntimeSession`. Server-side, that wipes just that episode's working directory and returns the fractional-GPU Ray actor to the pool, while the warm CUDA context, the shared `runtimeSessionId`, and the instance stay alive for the next episode (`sandbox_ray_farm.py:378-385`, `:219-226`).
- **Default / local:** `AgentCoreSandbox.__aexit__` → `client.stop()`; `LocalSandbox` → temp-workspace cleanup (`sandbox.py:194`, `:110`).

Why we do it, and why it matters. A session is not free to leave dangling: it holds a microVM (or an Instance's actor slot) and counts against AgentCore's **concurrent-session quota**, and the only automatic reclaim is the **~15-minute idle timeout** — far too slow at rollout throughput, where leaked sessions throttle concurrency and inflate cost. Explicit `StopRuntimeSession` releases the slot **immediately** instead of waiting out the idle clock, and closing the per-episode workspace guarantees the next episode starts from a **clean, isolated env** (no cross-episode file leakage). For the warm-shared path the *asymmetry is the whole point*: closing

only the `sub_session` frees the episode's env and hands the next episode a fresh workspace **without re-paying the ~60–80 s instance cold-boot** — whereas stopping the shared session would defeat warm reuse (re-pay the boot every episode) and could tear the instance out from under sibling episodes still running on it. So the single "always end at episode end" discipline yields two correct behaviors: **kill the per-episode session** where the session *is* the instance, and **kill only the sub-session** where one warm instance is shared. (The complementary hazard — a session lost *mid-episode* to the idle clock during a long weight-sync pause — is what the `HEARTBEAT=1` `SessionKeeper` addresses.)

5.6 One harness, many task families

The episode (`mini_claude_episode.py`) is **task-agnostic** — the same seed → agent loop → hidden check → reward cycle serves every family; only the *prompt, seed files, and checker* change (`task_families.py`), dispatched by `metadata.task.family`:

```
for name, content in task.setup_files():                    # seed the workspace
    await sb.write_file(name, content)
res = await harness.run(sb, session_id=sid,                 # mini_claude loop INSIDE
↪  the session:
                        adapter_url=session_url,            #   write_file / bash /
↪  read_file tools,
                        prompt=task.initial_user_prompt()) #   model turns dial the
↪  adapter back
exit_code, stdout = await _hidden_grade(sb, task)           # hidden checker, after
↪  the loop stops
reward = task.reward(0 if exit_code == 0 else 1)            # binary 1.0 / 0.0
samples = await adapter.finish_session(sid, base_sample=sample, reward=reward)  #
↪  exact tokens
```

Datasets are built once and seeded per episode: each row carries `metadata.task` (family + spec), reference answers are dropped so they cannot leak into prompts, and `generate.py` rebuilds the `Task` from that metadata. The same code runs the checker for MBPP, SWE, SQL, math, tool-use, RAG, CTF, and the rest — the sandbox is the only thing that changes.

5.7 Reading the episode shape into a config

The families of §“Tasks frontier labs run in a sandbox” differ mostly in their **episode shape** — turns per episode, response length per turn, total episode length — and that shape is what you size the training config against. Two axes drive every knob. **Turns per episode** sets `EPISODE_MAX_TURNS` and, multiplied by the per-turn response, the *sequence length* the trainer and inference engine must hold: single-turn families (math, instruction following) need only a long `--rollout-max-response-len`; long-horizon ones (SWE, web/computer use, multi-agent) need a large **context window** and a **session that stays warm for the whole episode**. **Where the tokens are** sets which side you scale (§“Scaling the loop”): generation-heavy families (math, reasoning code) are *inference-bound* — long responses, trivial checker; observation-heavy families (SWE, RAG, web, tool use) push cost into *context growth and the sandbox*. Rule of thumb: budget context ≈ `max_turns × (response_per_turn + observation_per_turn)`, size `--rollout-max-response-len` to the per-turn response at the high end (reasoning traces), and set rollout-buffer memory from the total episode length so long-horizon samples do not overflow it.

5.8 The episode, end to end

Taking one MBPP episode:

1. `run.sh` sets `SANDBOX=agentcore-runtime`, `DATASET`, `CONC`, `HARNESS_WORKERS`, then `ray job submit ⋯ train_async.py --custom-generate-function-path ⋯generate.generate`.
2. slime hands a prompt to `generate()`, which rebuilds the Task and queues the episode on the harness tier; a free dispatch worker opens `make_sandbox("agentcore-runtime")` → **cold-start** the session’s microVM (or takes a pre-opened one), then takes the least-loaded SGLang engine.
3. **Seed:** `write_file` the task’s files (`PROBLEM.md`, …) and the stdlib loop runner.
4. **Agent loop, inside the session:** each model turn dials back to the engine’s adapter (`[sglang-gen]`), each tool call (`write_file` / `bash` / `read_file`) runs locally in the session; the trainer long-polls the runner (`[sbx-op]`).
5. After the loop stops, the hidden checker runs in the session: `exit_code == 0` → solved; **reward = 1.0 / 0.0**.

6. The adapter's token record (`finish_session`) → one loss-masked `Sample` → buffer → Megatron step → weight sync to SGLang.

CPU → GPU is one env var: a CPU Runtime ARN (microVM, MBPP) → a GPU-Runtime ARN (Instances, KernelBench, `allclose` on cuda) — **zero change to the loop**.

5.9 Scaling, sharing, and network isolation

- **Scaling** follows §"Scaling the loop": grow the side that dominates the per-op latencies, and balance with the harness tier: a pull queue of dispatch workers, the least-loaded SGLang engine per episode (sticky for its turns), and a pool of sandbox Runtimes round-robined sticky per episode.
- **Sharing a GPU.** One session = one GPU, with no cross-session time-slicing [D5]. To pack several episodes onto one physical GPU you either run **many agents in one session** (they share the instance — no isolation) or run **your own fractional scheduler** (this project's `RayGpuFarm`, `num_gpus=0.5` actors, with an `nvcc flock` because a non-MIG L4 wedges under compile storms). Isolated scaling means *more instances*; there is **no MIG on L4**.
- **Warm-instance reuse — only for the Instances, and not used today.** A managed **microVM** cold-starts in a second or two, so the Terminal-Bench demo **does not reuse it**: every episode mints its own session, its own fresh microVM, which is also perfectly isolated. The **Instances** compute type is the opposite: a capacity-provider `g6` or `m5` takes **~60–80 s to cold-boot**. The removed Instance demos therefore tried one shared `runtimeSessionId` (one warm instance, cold boot paid once) plus a per-episode `sub_session` with a fresh environment. Measured on hardware, this cut `m5` cold boots **201 → 1** over a 10-step run, and a fractional-GPU Ray farm on a `g6` worked in the same shape (§"History: removed demos (2026-09-30)"). Detail: `docs/sandbox-fresh-env-on-persistent-instance.md`.
- **Network isolation — pull, don't expose.** Every Runtime is *reached* by an outbound SigV4 `invoke_agent_runtime`; **no Runtime has an inbound port**. The one connection the other way stays inside our VPC: the in-session agent loop dials the trainer's adapter ports, admitted SG→SG. IAM/SigV4 is the auth; the `session_id` is routing, not auth. For `networkMode=VPC` the Runtime has no internet route, so it needs interface endpoints (`ecr.api`, `ecr.dkr`, `s3`, `logs`, `sts`, `bedrock-agentcore`), and the ENI must land in an AZ that serves `bedrock-agentcore` — miss one and create fails with a null reason [D10, D11].

5.10 Can AgentCore host every frontier sandbox?

Mapping the task families of §“Tasks frontier labs run in a sandbox” onto AgentCore, the only hard limit is GPU **fractioning** — not any missing capability (✓ native ·◑ needs a BYO container / egress):

Task / sandbox	✓?	How — and the limit
Code exec + unit tests (DeepSeek, Qwen) [2,3]	✓	Code Interpreter, or your image on a Runtime
Math / logic verifier	✓	stdlib checker in the CPU microVM
Tool-use env (Kimi K2) [5]	✓	Python tools; MCP via Gateway
Per-repo Docker image (SWE) [6]	◑	Runtime runs your OCI image, isolated per session — but the managed microVM is **arm64-only** and caps the image at **2 GB** (§3); slim it, defer installs to `CMD`, or use the amd64 Instances substrate
Browser / computer use	✓	managed Browser tool
Networked RAG / web read	◑	egress — public, or VPC + endpoints [D11]
GPU kernel / CUDA (KernelBench) [7]	◑	Instances only; 1 session = 1 GPU; **no MIG on L4**

5.11 Related work — DeepSeek DSec, a purpose-built sandbox platform

DSec (DeepSeek Elastic Compute) [31] is DeepSeek’s in-house sandbox platform for agentic RL, evaluation and environment building. It has served every sandbox workload from DeepSeek V3.2 to V4.1. It is the best published reference point for

what a sandbox layer looks like when it is built for RL rather than rented. One unit is about 160 CPU nodes with 30K cores. It serves about **3M sandboxes a day**, with peaks of **380K running at once** and **more than 5,000 created per second**.

Its design choices line up closely with the questions this report asks of AgentCore:

- **Four backends behind one SDK:**
  - warm pre-created containers for short stateless calls, including GPU kernels on MIG slices;
  - containers for SWE and tool use;
  - Firecracker microVMs for security-sensitive and computer-use tasks;
  - QEMU VMs for Android and GUI tasks.

  The caller picks the backend, which is the same idea as our three sandbox tiers.
- **Layered images, loaded on demand.** Base OS, task workspace and toolkit are versioned separately (11K bases × 100K workspaces in one week) and stacked at start time. They are read on demand from DeepSeek's shared filesystem: a sandbox reads only 4–13 % of its image, and on an 8,192-container burst a full pull took 1.71× longer.
- **Heavy overcommit.** About 90 % of sandboxes use at most 5 % of the CPU they request, so one node runs about 3,200 containers or 800 microVMs. Latency-sensitive sandboxes get protected CPU scheduling.
- **Coupled to the trainer.** The agent loop runs on the sandbox platform, outside the preemptible GPU job. When a job is preempted, its sandboxes are paused (containers frozen, microVMs snapshotted) and resume on the next request.
- **Reward-hacking controls.** Their agents forged requests to the in-sandbox shell service, overwrote `/bin/bash`, port-scanned for code mirrors and crashed the kernel through `/proc`. The fixes are AppArmor rules plus per-sandbox network allowlists that can change between task stages.

	DSec [31]	AgentCore (this account, us-west-2, 2026-09-26)
Sandbox creation rate	> 5,000 /s	**25 /s** new Runtime sessions (adjustable)
Running at once	380K	5,000 Runtime sessions · 1,000 Code Interpreter sessions

	DSec [31]	AgentCore (this account, us-west-2, 2026-09-26)
Distinct environments	11K bases × 100K workspaces, composed at start	one image per Runtime; max **1,000** Runtimes per account
Image delivery	on demand; start time barely depends on image size	full image per cold boot (≈ 0.8 s stdlib, ≈ 15 s at 447 MB, grows with size); 2 GB microVM cap. **Ours:** no image at all on Code Interpreter (pandas/scipy/sklearn/torch-cpu preinstalled); install heavy deps at boot to fit 2 GB; > 2 GB on a Runtime Instance (60–80 s m5, ≈ 207 s g6 cold boot, paid once through a warm-shared session)
Hiding the cold boot	not needed: lazy loading keeps the pull time low; pre-warming rejected, since DSec's bottleneck is I/O volume, not latency	**pre-open (ours)**: while the current batch runs, the sandboxes of the next 1–2 batches are opened (image pulled, microVM booted), which hides the image boot time; median wait for a ready sandbox 774 ms → **5 ms** (warm-start probe, small stdlib image), makespan −20 %
Sandbox lifetime	p50 15–17 min, p99 > 3 h	15-min idle timeout by default; max lifetime 8 h on a microVM, 14 days on a Runtime Instance

	DSec [31]	AgentCore (this account, us-west-2, 2026-09-26)
Survives a trainer stall	pause/resume, state kept	**no pause/resume, no state kept**; a heartbeat keeps the sandbox alive
Agent loop	outside the GPU job	**outside the trainer too (ours, every demo since 2026-09-28)**: the mini_claude loop + its tools in the episode's own Runtime session, one loop per sandbox (the removed `demo_cpu_branching` ran the Claude Code CLI there); the trainer keeps a dispatch tier + one adapter per engine. Results committed before 2026-09-28 used the in-trainer loop
Reward-hack controls	AppArmor + network allowlists	microVM boundary; VPC mode (a Landlock + no-TCP jail on a shared `m5` pool was measured, then removed)

The headline gaps are the three things AgentCore does not expose: image layers, session pause, and a creation rate in the thousands. One number bears directly on this project. DSec's median sandbox lives 15–17 minutes, almost exactly AgentCore's default idle timeout, so bridging trainer stalls — now the default: heartbeat on for every remote own Runtime, plus a 3600 s idle timeout — (§"Hiding the container launch: pre-open and heartbeat") is the normal case rather than an edge case. Two DSec ideas can be copied cheaply on AgentCore: moving the agent loop out of the trainer process (now done in every demo: the loop runs in the sandbox session and scales with it, §"Scaling the loop"), and allowlisting the network per

task through VPC mode. We also adopted three of its metrics (batch time-to-ready, peak creation rate and concurrency compared with the quotas, and session lifetime compared with the idle limit); see §“The Terminal-Bench demo in depth” for how they are computed and what the first log shows.

In this project the policy is always **Qwen3-4B**, trained by slime, with AgentCore as the sandbox and the harness running inside the sandbox session. The next section applies all of the above to one demo, RL on Terminal-Bench (§“The Terminal-Bench demo”), and §“The Terminal-Bench demo in depth” gives its pipeline, metrics and measured results.

6 The Terminal-Bench demo

The demo lives in `code/demos/demo_terminalbench_kernelbench/` (it was `code/demos/demo_cpu_terminalbench/`, the microVM-only version described here, until 2026-10-06). It is a self-contained recipe: a vendored library (`agentcore_slime_demo/`), the Terminal-Bench tooling (`terminalbench/`), the deploy helpers (`deploy/`) and six numbered step scripts. The policy is **Qwen3-4B**, trained by **slime** with fully-async GRPO. **AgentCore is the sandbox**, and the **harness is the mini_claude loop, running inside each episode’s sandbox session**. It is **REMOTE-ONLY**: `SANDBOX=agentcore-runtime` is the only backend.

6.1 Which sandbox does a task need? Three tiers

Pick the **lowest tier that works**: move to tier 2 for a run-level need, and to tier 3 only when the task’s environment itself must change.

Tier	Sandbox	Use it when	In this project
1 ·Managed Code Interpreter	no image, no Runtime	**probes only**: a Code Interpreter session has no route back to the trainer's adapter, so the in-session agent loop cannot run there	warmstart probe
2 ·Own Runtime, generic image	own Runtime + a stdlib image	the task needs no task-specific environment (code, math, SQL, RAG, tool calls)	warmstart probe
3 ·Own Runtime per row, custom image	one image + one Runtime per task	the environment differs per row	**Terminal-Bench**

Terminal-Bench is tier 3 on the managed arm64 microVM fleet. The fleet's 2048 MB image cap cannot be raised, so tasks whose image does not fit are reported and left out.

6.2 Use case covered

Agentic RL where **every task needs a different OS image** (packages, services, files, compilers) and the agent changes the whole machine through the shell. Terminal-Bench [6c] ships each task as its own container: a task `Dockerfile`, a hidden pytest oracle (`tests/` + `run-tests.sh`) and a reference solution. This is the **"large image pool"** shape: hundreds of bespoke images, each served by its own AgentCore Runtime, with strict row → image → Runtime routing.

6.3 Why this demo

- **It is the hardest sandbox shape in the literature datasets.** No generic image can serve it, and a shared session would leak one episode's changes into the next. The reasoning, code, math, SQL and tool-use datasets all run on one generic image (§"Language and environment per family"), so they do not exercise the sandbox layer the way Terminal-Bench does.
- **It shows AgentCore scaling to a pool of ~218 Runtimes** built from the tasks' own Dockerfiles, with no per-task code: the row names its image, and `image_routing.py` maps image → ARN.
- **It shows the two latency fixes a per-episode microVM needs**: **pre-open**, which boots the next batches' sessions ahead of time, and a **heartbeat**, which keeps them alive across the weight-sync stall.
- **It exercises every tier of the loop at once**: the sandbox (a fresh microVM per episode), the harness (one loop per session, dispatched by the harness tier) and inference (the least-loaded SGLang engine per episode).

6.4 Design considerations

- **One fresh microVM session per episode** (`terminalbench_pool.py`): the tasks mutate the OS, so no state may cross episodes or GRPO siblings. The session is stopped when the episode is graded.
- **Wrapping, not rewriting, the task image.** `deploy_terminalbench.py` appends one layer to each task's own Dockerfile: a standalone Python (`/opt/acs/py`) and the stdlib AgentCore HTTP server. The task's own interpreter is never touched; Terminal-Bench images range from python3.8 to no Python at all.
- **Strict routing.** The episode's `task.image` (the task name) is looked up in the image → ARN map (`arn_by_image.json`, exported as `AGENTCORE_SANDBOX_RUNTIME_ARN_BY_IMAGE`). An unmapped task is an error. It is never sent to a fallback Runtime, which would run it on another task's image.
- **Look-ahead pre-open** (`tb_lookahead_source.py`, `TB_PREOPEN_BATCHES=2`): the data source peeks at the next two batches' rows, and the pool boots their sessions on the right task Runtime, so the cold start leaves the critical path.
- **Heartbeat** (`HEARTBEAT=1`, default): a `SessionKeeper` pings every idle pre-opened session below the microVM idle timeout, so a trainer stall cannot re-

claim it. A reclaimed session id comes back as an empty sandbox, and nothing reports the loss.

- **Harness in the session, harness tier in the trainer.** The mini_claude loop runs inside the session and dials the trainer's per-engine `AnthropicAdapter` back over the VPC. `HARNESS_WORKERS` dispatch workers (default `RB × NS` = 32) pull episodes off one queue. Each episode takes its session first, then the least-loaded SGLang engine, which stays sticky for all its turns.
- **Oracle validation before training.** `validate_oracle.py` checks each task on its own Runtime: an untouched environment must fail and the reference solution must pass. Tasks that fail the check are dropped from the data.
- **What doesn't fit.** Of 229 single-container tasks, **7 fail to build for arm64** and **4 exceed the 2 GB microVM cap**. They are listed in `terminal-bench/tasks/images.csv` and left out of the data. Multi-container tasks are skipped, since one image maps to one sandbox.
- **Network**: VPC mode with NAT egress (`TB_NETWORK=VPC`). The loop dials the adapter back through an SG→SG rule on ports 18100-18163, and `run-tests.sh` installs uv + pytest from the internet.
- **Names shared with nothing else**: Runtimes are `acs_tb_<task>` and ECR repos are `acs-tb-<task>`.

Isolation. One episode = one microVM = one session. Every episode gets a separate kernel, and nothing is shared between episodes except the task's read-only image. The adapter port hole (SG→SG) is the only inbound path.

6.5 The pool

`bash 2_build_image.sh` fetches the corpus (`fetch_terminalbench.py`) and builds one arm64 image per task with `docker buildx`, then pushes it to ECR `acs-tb-<task>`. The step is resumable and takes about 2 h cold. `bash 4_deploy_runtime.sh` registers one VPC Runtime per image and writes the map `arn_by_image.json`. The files named below live in `terminalbench/tasks/`. Measured on 2026-09-30:

Stage	Count	Source
single-container tasks in the corpus	229	`images.csv`

Stage	Count	Source
failed to build for arm64	7	same
over the 2 GB microVM cap	4	same
Runtimes registered	**218**	`arn_by_image.json`
oracle-usable (untouched fails, reference passes)	166	`oracle.csv`
dataset rows (train / eval, split by task)	134 / 31	`data/` (train, eval)

Cold start of all 218 Runtimes (`probe_coldstart.py` → `coldstart.csv`, 1,090 opens). A first open of each Runtime, one at a time, took p50 **0.60 s**, p90 0.96 s and max 3.3 s. A burst of four concurrent opens per Runtime took p50 **4.7 s**, p90 12.3 s and max 49 s. 1,085 of 1,090 opens succeeded; the other 5 were Runtime health-check or initialization timeouts. So the pool opens fast one at a time, and a burst is what pre-open has to hide.

6.6 Hiding the container launch: pre-open and heartbeat

A fresh session per episode means a cold start per episode: allocate a microVM, pull the task's image, boot it. The demo hides it with two mechanisms, both on by default.

Pre-open the next one or two batches. slime hands the pool only `CONC` × `engines` episodes of the current batch, and draws the next batch only when this rollout ends, so the pool alone cannot see far enough ahead. A **look-ahead data source** (`tb_lookahead_source.py`, wired with `--data-source-path`) replays slime's deterministic prompt order: a shuffled dataset walked in order, with a fixed seed per epoch. With the fully-async driver it slides per group. Drawn groups are appended to the pool's demand, the next `TB_PREOPEN_BATCHES` × `RB` prompts become the look-ahead, and aborted groups are un-reserved. The pool keeps that many upcoming episodes booted, `TB_PREOPEN_BATCHES` × `RB` × `NS` = 2 × 4 × 8 = **64**, each on its own new session on its task's Runtime. When a worker claims one, the next

upcoming episode starts booting. A misprediction is trimmed at the next draw and those episodes open on demand. `TB_PREOPEN_BATCHES=0` turns it off.

Heartbeat across the weight-sync stall. While the trainer syncs weights into SGLang, generation pauses and sessions receive no invokes. An own-Runtime session that stays idle past its timeout is reclaimed. A `SessionKeeper` (`session_keeper.py`) registers every open session and pings (`exec("true")`) any session that has been idle for longer than `WARM_PING_FRACTION` × `WARM_IDLE_TIMEOUT` (0.5 × 900 s). The keeper lives **outside** the harness, so no harness needs its own keepalive. As a backstop, Runtimes created by `deploy/agentcore_deploy.py` get an idle timeout of `RUNTIME_IDLE_TIMEOUT` = 3600 s. The cost is that an abandoned session is billed for up to an hour.

The `terminalbench/warmstart/` probe isolates both mechanisms (`bash terminalbench/warmstart/run_warmstart.sh`: host-only, no GPU, no model, about 7 min and a few cents). The same M = 24 MockPolicy `sql` episodes run in batches of 6 (think time 2 s) against a stdlib Runtime `acs_warmstart_cpu` with a 60 s idle timeout, and against the Code Interpreter. Recorded run `warmstart_20260925_100103.*` (2026-09-25, us-west-2, all rewards 1.0):

Backend	Mode	Wait p50	Episodes that waited	Idle held	Makespan
own Runtime	cold	774 ms	24/24	0 s	31.9 s
own Runtime	pre-open	**5 ms**	8/24	75 s	25.5 s (−20 %)
Code Interpreter	cold	1715 ms	24/24	0 s	28.7 s
Code Interpreter	pre-open	**0 ms**	6/24	70 s	24.3 s (−15 %)

Backend	Heartbeat	Sessions surviving 150 s idle	Resume
own Runtime	on (14 pings)	**3/3**	104 ms, files intact

Backend	Heartbeat	Sessions surviving 150 s idle	Resume
own Runtime	off	0/3	**silently reclaimed**: the same session id returns a fresh, empty sandbox (948 ms)
Code Interpreter	on or off	0/3	expires at its session timeout regardless of pings

What the probe shows:

- **Pre-open takes the boot off the critical path; only the first batch pays it.** The cost is idle held time, meaning booted sessions waiting for work. The Terminal-Bench smoke shows the same trade-off at full scale (about 95 % of pre-opened time idle), which is why `TB_PREOPEN_BATCHES=1` is worth trying there.
- **A heartbeat only helps an own Runtime.** Without it, a reclaimed session *looks healthy* but has lost its files. Because the failure is silent, the demo keeps the heartbeat on by default.
- **On the Code Interpreter a heartbeat does nothing.** A session ends at the `sessionTimeoutSeconds` set when it starts, however much traffic it gets.

6.7 Scaling the harness with inference and the sandbox: the harness tier

Rollout throughput is `min(inference, sandbox, harness)` (§“Scaling the loop”). The harness has to scale **along with** the other two, or a few coroutines silently become the ceiling. It should also scale **with the sandbox rather than inside the trainer**: the mini_claude loop runs in the episode’s own session, so N sessions carry N loops, as DSec does [31]. `TerminalBenchPool` subclasses the shared `HarnessPool` (`harness_pool.py`) and swaps only the sandbox tier. The three tiers are

sized independently and joined per episode, and no worker owns an engine or a session:

- **harness**: `W = HARNESS_WORKERS` dispatch workers in the trainer (default `RB × NS = 32`). A free worker pulls the next episode off one shared queue.
- **sandbox**: a fresh session per episode on its task's Runtime (pre-opened, heartbeated).
- **inference**: the engine with the fewest in-flight episodes (ties rotate), sticky for all the episode's turns. This replaced the earlier four workers, each pinned to one engine.

`CONC` defaults to `[2W / E]`, so W running and W queued episodes are admitted. The sandbox is taken before the engine, so an episode waiting for a session does not count as engine load. Each episode returns one bare `Sample`, the shape the fully-async driver needs. Each episode's `[demo]` record shows which tier binds: `qdepth > 0` (harness), `sandbox_wait_ms > 0` (sandbox), or a climbing `engine_inflight` (inference). A session that cannot host the loop (no `python3`, or the adapter unreachable) raises `HarnessPlacementError`. There is no in-trainer fallback.

6.8 Run it step by step

Run the numbered scripts from `code/demos/demo_terminalbench_kernelbench/` **on the host**; the container steps `docker exec` into `slime_trainer` themselves.

```
bash 2_build_image.sh      # fetch the corpus (once) + build/push one arm64 image
↪  per task (resumable, ~2 h cold)
bash 3_push_ecr.sh         # report only (step 2 pushed)
bash 4_deploy_runtime.sh   # one VPC Runtime per image ->
↪  terminalbench/tasks/arn_by_image.json
bash 1_prepare_data.sh     # rows for the deployed tasks only (train/eval split by
↪  task)
bash 5_run.sh              # 10-step smoke -> logs/<tag>.{log,csv,eval.csv,png}
bash 6_cleanup.sh          # delete every acs_tb_* Runtime (ECR images kept)
```

Setting `TB_LIMIT=2` (or `TB_TASKS=a,b`) on steps 2 and 4 validates the pipeline on a couple of tasks first, and `validate_oracle.py` checks each task's oracle on its Runtime. The long run is step 5 itself: `bash 5_run.sh` defaults to `NR=epoch`, one pass over the training split (251 rows / `RB=4` = 62 train steps), with eval every `EVAL_INTERVAL=10` steps and one final checkpoint. It drives the trainer **inside `slime_trainer`**, after steps 1-4. Never run two `run.sh` at once: the second one's

Ray start kills the first one's job. Training starts from a base Megatron checkpoint in `code/p4_slime_train/ckpt/`: `Qwen3-4B_torch_dist` by default, `Qwen3.5-9B_torch_dist` with `MODEL=qwen3.5-9B` (build it once with `prepare_qwen35.sh`; override either with `CKPT_BASE`).

Knob	Default	Meaning
`NR` / `EVAL_INTERVAL`	11 / 10 (`5_run.sh`)	rollouts (NR−1 train steps) / eval cadence
`RB` / `NS`	4 / 8	prompts per step / GRPO group size
`TB_PREOPEN_BATCHES`	2	batches of sessions booted ahead (0 = open on demand)
`HEARTBEAT`	1	keep idle sessions alive across trainer stalls
`EPISODE_MAX_TURNS` / `EPISODE_MAX_TOKENS`	8 / 1024	agent turns / tokens per turn
`TB_EXEC_TIMEOUT`	180	per bash-tool command, seconds
`TB_BUILD_CONC` / `TB_REBUILD`	16 / 0	parallel image builds / force rebuild

7 The Terminal-Bench demo in depth — pipeline, metrics, and measured results

This section is the deep dive behind §"The Terminal-Bench demo": the pipeline and its configs, the metric set, how to size the two episode knobs, the measured smokes, and the plan to convergence. The numbers are read from this project's own run artifacts: the per-step CSVs and the `[sbx-op]`, `[sglang-gen]` and `[demo]` records in the logs. They come from **short runs** (10-step smokes), so treat them as the shape and order of magnitude of the loop, not as converged results. Where a metric is only *instrumented* (emitted by the code but not yet populated in a run), this is said explicitly.

7.1 The pipeline — one episode, one step

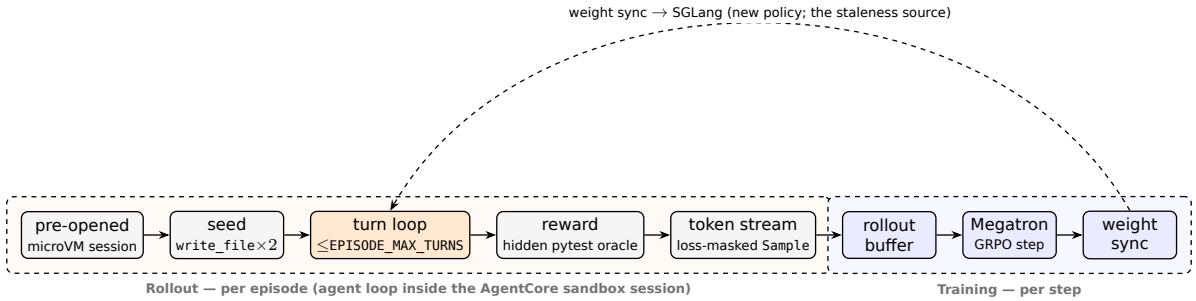


Figure 8: The Terminal-Bench pipeline. An episode takes its pre-opened session on its task's Runtime, seeds the workspace, then runs the **turn loop** — each turn a generation through the proxy (`[sglang-gen]`) followed by a tool exec in the sandbox (`[sbx-op]`). The task's hidden oracle gives the reward; `build_token_stream` emits a loss-masked `Sample` into the rollout buffer; Megatron takes a GRPO step and **weight-syncs** the new policy back to SGLang — the point at which rollouts generated on an older policy version become *stale*. The two labeled latencies are the two things the scaling decision (§"Scaling the loop") is made from.

Architecture. The topology is the one in §"The call path". slime's `generate()` hook hands each episode to the harness tier, which takes the episode's pre-opened session on its task's Runtime and then the least-loaded engine. The agent loop runs inside the session. Each **model turn** dials back to the proxy, which returns the exact sampled tokens and logprobs (the TITO property). Each **tool call** (`bash` / `write_file` / `read_file`) runs in the session. After the loop stops, the hidden oracle produces the reward.

The four configs: trainer, inference, harness, sandbox.

Component	Where it runs	Config
Trainer	4 GPUs on the 8×L40S box	Megatron-LM, GRPO, fully-async rollout; RB=4 × NS=8 = `global_batch_size=32`; `ACTOR_GPUS=4`

Component	Where it runs	Config
Inference	4 GPUs on the same box	SGLang, 2 engines (`GPUS_PER_ENGINE=2`), qwen25 tool-call parser + qwen3 reasoning parser
Harness	mini_claude inside the session; dispatch tier + adapters in the trainer	`HARNESS_WORKERS=32`, least-loaded engine per episode, `CONC = ⌈2W/E⌉`; ≤ 8 turns × 1024 tokens per turn
Sandbox	218 AgentCore arm64 microVM Runtimes, VPC + NAT	one fresh `runtimeSessionId` per episode on the task's Runtime; `TB_PREOPEN_BATCHES=2`, `HEARTBEAT=1`, `TB_EXEC_TIMEOUT=180`

This is the disaggregation of §"Agentic RL and its reference architecture" made concrete. The GPUs go to training and inference, the harness rides in the sandbox session it drives, and the sandbox is the isolated remote seam.

Step by step (one episode).

1. Take the episode's pre-opened session, or cold-start one on the task's Runtime.
2. Seed the task prompt and the stdlib loop runner (`session_runner.py`).
3. Run the turn loop (≤ `EPISODE_MAX_TURNS`): generate through the proxy (`[sglang-gen]`), then run the tool in the session.
4. After the loop, inject the oracle (`tests/` + `run-tests.sh`) and run it.
5. Compute the reward: 1.0 if every oracle test passes, else 0.0.
6. The adapter's token record becomes a loss-masked `Sample`, which goes to the rollout buffer, then a Megatron GRPO step, then a weight sync back to SGLang.

Lifecycle. One session = one episode. The first invoke cold-starts the microVM, and each further op under the same `runtimeSessionId` reaches the same warm VM. `StopRuntimeSession` ends the session when the episode is graded. The idle timeout

(~15 min by default, 3600 s on Runtimes created by `deploy/agentcore_deploy.py`) and the 8 h max lifetime are only fallbacks. Under asynchronous training, the weight-sync pause could trip the idle clock of a pre-opened session; the heartbeat prevents that.

What is measured, and what is only instrumented.

Metric	Meaning	Status
`rollout/raw_reward`, `eval/terminalbench`	mean oracle reward per step; held-out solve rate	**measured**
`[demo] episode_end` fields	per episode: `solved`, `tests_passed`, `tests_total`, `turns`, `tool_calls`, `stop`, `agent_ms`, `grade_ms`, `harness_where`, `infra_error`	**measured**
harness-tier fields	`worker`, `qdepth`, `engine`, `engine_inflight`, `sandbox_wait_ms`	**measured**
pre-open fields	`open_wait_ms`, warm hits, `preopen_idle_ms`	**measured**
`perf/sandbox_*`, `perf/sglang_gen_*`	warm data-op vs session-open latency; per-turn generation latency	**measured**
`perf/step_time`, `wait_time_ratio`, `actor_train_tok_per_s`	step wall-clock; fraction of the step the trainer waits on rollout; trainer throughput	**measured**
`ready_span_s`, `open_peak_per_s`, `sessions.csv`	batch time-to-ready, peak opens per second vs the 25/s quota, session lifetime vs the idle limit (after DSec [31])	**measured** (re-parsed from `[sbx-op]`)

Metric	Meaning	Status
staleness, discard ratio, importance-sampling ratio	off-policy signals	**instrumented or configured only**, no value recorded

`metrics.py` parses the log into a per-step CSV (`<tag>.csv`), a per-eval CSV (`<tag>.eval.csv`), a `[demo]` summary (`<tag>.demo.csv`) and a PNG. Sandbox latency is split into **warm data ops** and **session opens**, and a cold boot is counted only when an open exceeds `COLD_BOOT_MS` (5 s). CPU and memory used versus requested, DSec's overcommit signal, is **not measured yet**: it needs per-exec resource usage from inside the sandbox server.

7.2 Sizing the config — response length per turn, and turns per episode

Max response length per turn (`EPISODE_MAX_TOKENS`, 1024; `--rollout-max-response-len`). Set it from the p95 of the policy's own per-turn response for the family. **Too small** truncates the turn mid-thought: the 2026-09-30 smoke stopped 115 of 449 episodes on `max_tokens`. **Too large** wastes KV cache and slows decode. `MAX_TOKENS_PER_GPU=16384` must hold one full trajectory: 8 × (1024 generated + ~600 tool tokens) plus the prompt. Each tool result is clipped to `MINI_AGENT_MAX_TOOL_CHARS=2000`, which bounds the trajectory.

Max turns per episode (`EPISODE_MAX_TURNS`, 8). This cap is the single biggest lever on episode cost. Each extra turn adds one generation and one sandbox round trip, and it grows the context for every later turn. More turns raise the ceiling on reward, but they lengthen episodes, which under asynchronous training means **more staleness** and longer-held sessions. Terminal-Bench is a long-horizon family, and 8 turns is a smoke-sized budget. Both knobs are env vars read by `run.sh`, so no redeploy is needed to change them.

7.3 Measured — the 2026-09-23 smoke (first folder)

This run used the first `demo_cpu_terminalbench` folder (removed 2026-10-06), before the 2026-09-28 merge (`_smoke_logs/lanes_tb_0923/terminalbench.log`). It

ran 382 episodes in ~70 min, ended `Job succeeded` and had 0 infrastructure errors.

- **Cold start fully hidden.** `open_wait_ms` p95 was 0, the open p50 was 0.66 s, and 100 % of opens were warm hits.
- **The price: idle pre-opened time.** `preopen_idle_ms` p50 was 7.3 min against an episode p50 of 21 s, so about **95 % of the pre-opened microVM time was idle**.
- **Sessions.** Re-parsed, the run had 414 sessions, with a lifetime p50 of **471 s** and a p95 of **1841 s**. **55** of them outlived 900 s: the heartbeat kept them alive, and every open pinged back healthy. Opens peaked at **36 per second**, above the 25/s quota, with **no throttling** observed. This is an observation, not an AWS statement.
- **Sparse reward for a 4B policy.** 4 of 382 episodes were solved and `eval/terminalbench` was 0.0, though 42 % of episodes passed at least one test.

7.4 Measured — the 2026-09-30 smoke (current folder)

The 10-step smoke `bash 5_run.sh` started at 13:48 UTC on 2026-09-30 (`logs/run-agentcore-runtime_20260930_134858.log`). **The run was killed on purpose at 14:16, after 9 of its 10 train steps**, to free the box for the Terminal-Bench-only cleanup. There was no final rollout, no eval and no CSV, so the numbers below are read from the log. `rollout/raw_reward` per step was 0, 0.16, 0, 0, 0.25, 0, 0, 0.09, 0, and `perf/step_time` was 30-41 s in steady state (140 s and 136 s on the first step and on a step held up by one 504 s exec). Over 460 `[demo] episode_end` records, **19 episodes were solved** and 216 passed at least one oracle test, with **0 infrastructure errors**. 449 ran their loop in the session (`harness_where=session`). Stop reasons were the turn budget 175, `end_turn` 159 and `max_tokens` 115, and `agent_ms` had a p50 of 12.2 s and a p95 of 57.8 s, with a turns p50 of 6. The 61 tracebacks in the log are all `asyncio.CancelledError` from the fully-async driver cancelling surplus groups, which is benign. `train/pg_loss` was 0 on most steps because most GRPO groups had zero reward variance. The log tail shows slow execs with transient `RuntimeClientError` retries. **A re-run of the 10-step smoke is needed** before convergence run 20.

7.5 Measured — the 2026-09-30 re-run (current code)

`bash 5_run.sh` re-run at 15:31 UTC (`logs/smoke_tbonly-agentcore-runtime_20260930_15311` completed 10 train steps and the final eval in **22 min** (the 2026-09-23 smoke took about 70 min). There were 449 episodes (387 train including the fully-async over-draw, 62 eval) with **0 `infra_error`**. 22 of 387 train episodes and **1 of 62 eval** episodes were solved (`eval/terminalbench` = 0.016). Per-step `rollout/raw_reward` was 0, 0.09, 0.09, 0, 0.25, 0, 0, 0.09, 0, 0.16. `perf/step_time` p50 was 40 s. No train episode waited for a boot (pre-open), and one heartbeat ping was needed (pre-open idle max 194 s). Train episode p50/p90: agent loop 12.1/36.2 s, grading 8.9/20.6 s, whole episode 24.9/71.7 s. Only 4 of 49 GRPO groups had a nonzero-variance reward. One bug was fixed to get here: eval hung after its last episode because `get_tb_pool` was a process-wide singleton bound to the fully-async worker's event loop, so eval's futures were resolved from the wrong thread. It is now one pool per event loop.

7.6 Measured — Qwen3.5-9B, one full epoch (2026-10-11)

`MODEL=qwen3.5-9B bash 5_run.sh` (`logs/epoch-9b-hybrid_20261011_070427.{csv,eval.csv`) trained **all 62 steps of one epoch** in 4 h (07:04-11:04 UTC, including a 25 min final save) at `lr` 1e-6, on the same 4 train + 4 inference L40S GPUs as Qwen3-4B. Of 3,636 episodes, 13 had an `infra_error` (8 grading time-outs, 3 AgentCore 502s, 2 empty episodes); all were dropped without stopping the run.

	Qwen3-4B (10-step smoke)	Qwen3.5-9B (epoch, lr 1e-6)
Step time	~1.5 min	~2.5-3 min
Training reward, first / last 10 steps	0.20 (10 steps)	0.27 / 0.29
GRPO groups with all-equal rewards (no gradient)	21 of 40	137 of 248 (55 %); 7 steps with `grad_norm` = 0
Eval, Terminal-Bench microVM (31 tasks x 2)	0.0	0.10-0.15 at steps 10-60
Eval, KernelBench (20 x 2)	0.425	0.21-0.35

	Qwen3-4B (10-step smoke)	Qwen3.5-9B (epoch, lr 1e-6)
Eval, Terminal-Bench m5 (4 x 2)	0.0	0.0

A higher learning rate does not help. `MODEL=qwen3.5-9B LR=3e-6 NR=21 bash 5_run.sh` (`logs/lr3e-6-9b-hybrid_20261011_140649.*`) trained 20 steps from the same base checkpoint (1 h 38 min, 2 `infra_error` in 1,267 episodes). Eval at steps 10 / 20 was 0.13 / 0.10 on Terminal-Bench microVM and 0.35 / 0.39 on KernelBench, the same as at 1e-6 within noise: both runs always solve the same four tasks (`fix-pandas-version`, `jq-data-processing`, `swe-bench-langcodes`, `vertex-solver`) and differ only in a fifth. Training reward fell from 0.28 (first 10 steps) to 0.20 (last 10), and `train/kl_loss` ran 4-8x the 1e-6 run's at the same step, ending at 0.38 at step 20 (the KL loss coefficient is 0, so nothing holds the policy near the reference). 52 of 80 GRPO groups had all-equal rewards. So the limit is the reward signal, not the step size: keep 1e-6 (or add a KL coefficient before going higher) and work on the share of groups with a gradient.

What it shows. The larger policy is what moves Terminal-Bench off zero: 5 of 31 held-out tasks solved at step 10, against none for 4B. It does not show learning within the epoch: training reward rises only from 0.27 to 0.29, and eval is flat within its noise (one episode = 0.016 on the microVM eval, 31 tasks x 2). At 1e-6 and 32 episodes per step the policy barely moves; raising the learning rate is the obvious next lever, and it did not help (next paragraph). The eval sets are small, so read eval changes of a few hundredths as noise.

What 9B needed on L40S. Each of these failed a run before it was fixed, and each is now set by `MODEL=qwen3.5-9B` in `run.sh`:

- **Thinking off by default.** slime's `AnthropicAdapter` renders the chat template without `enable_thinking`, and Qwen3.5 ignores the `/no_think` switch, so a `<think>` trace eats the 1,024-token turn. `prepare_qwen35.sh` makes a copy of the HF directory whose template defaults thinking off.
- **The `qwen3_coder` tool parser.** Qwen3.5 writes tool calls as XML (`<function=...><paramet`) not Qwen3's JSON (qwen25).
- **Two HF directories.** With the vision `preprocessor_config.json` present, slime loads Qwen3.5's image processor and rejects text prompts; without it,

SGLang cannot start the model. slime's `--hf-checkpoint` has no processor files and `--sglang-tokenizer-path` points at a sibling that has them.

- **Unfused attention.** The full-attention layers have head_dim 256: FlashAttention-2 has no backward pass for it below sm80/sm90 and cuDNN fused attention refuses it on the L40S (sm89), leaving Transformer Engine's unfused path. Its memory is quadratic and it pads every sequence of a packed micro-batch to the longest one (one 6k sequence: 1 GB; the same packed with three 2k sequences: 3.9 GB). `MAX_TOKENS_PER_GPU=8192` ran out of memory at step 46 of the first epoch attempt; 4096 gives every sequence over 4k a micro-batch of its own.
- **TP=4 and Adam on the CPU.** At TP=2 the bf16 weights plus fp32 gradients alone are about 27 GB per GPU; TP=4 over the 4 trainer GPUs with `--optimizer-cpu-offload` leaves about 19 of 44 GB in use.

Checkpoints are large. A 9B checkpoint with optimizer state is about 200 GB, and the root EBS volume writes at its 125 MB/s gp3 baseline, so the final save takes about 25 min, with the writer processes in `D` state (it looks hung but is not). The instance's local NVMe (`/opt/dlami/nvme`, 6.9 TB) would be faster but is not mounted in `slime_trainer`.

7.7 Experiment plan — training to convergence

A demo is "done" only when it shows **learning to convergence**, not a smoke run. The convergence bar is:

1. Train enough steps that the **held-out `eval/terminalbench` rises and then plateaus**, with eval every `EVAL_INTERVAL` steps.
2. `rollout/raw_reward` and `eval/terminalbench` trend up and then flatten in the run's PNG.
3. `train/grad_norm` and `train/kl_loss` stay sane: the policy moves without diverging.
4. Only then apply §"Scaling the loop": scale the *measured* bottleneck and show the `perf/step_time` gain.

Run `MODEL=qwen3.5-9B bash 5_run.sh`	
Steps / eval	`NR=epoch` (62 train steps per epoch), eval every 10 (`EVAL_NS=2`)
Held-out start → target	`eval/terminalbench` 0.13 (9B, step 10) → a rise beyond the eval's noise
Measured bottleneck	sparse reward (55 % of GRPO groups flat at 9B) + a small learning rate
What to scale	not LR (3e-6 drifted without gain); first the reward signal (curriculum / partial credit), then epochs; more `HARNESS_WORKERS` + engines once it learns

Two levers matter most before scale:

- **Reward signal first.** The oracle reward is sparse (55 % of GRPO groups flat even at 9B). An easy-first curriculum (by the task's `difficulty` field) or a partial-credit reward from `tests_passed / tests_total` would give GRPO non-zero advantages on more groups.
- **Pre-open cost.** `TB_PREOPEN_BATCHES=1` halves the booted-ahead sessions. The 2026-09-23 smoke shows how much idle time the default of 2 buys.

8 History: removed demos (2026-09-30)

On 2026-09-30 the project was cut down to the Terminal-Bench demo. The rest is still in git history. Each line below gives what the removed item was for and what it established.

- **`demo_cpu_tasks`**: nine literature datasets (GSM8K, MATH-500, DeepScaleR, MBPP, HumanEval, Gretel SQL, HotpotQA, xLAM, MCP), all on one generic stdlib image. Its smokes showed that most families need no custom image: `eval/gsm8k` 0.90, `eval/math500` 0.89, `eval/mbpp` 0.56. Its `SCALE=1` preset (4 engines : 256 harness workers : 256 sessions, 592 episodes) was inference-bound, which is how the harness tier was proven.

- **`demo_gpu_single`, `demo_cpu_instance`**: one session per capacity-provider Instance (g6/L4 KernelBench; m5). The warm-shared KernelBench run scored `eval/kernelbench` 0.65. It also showed that an Instance costs ~90 s or more of cold start (torch pip-installed to stay under the image cap), and that there is no usable GPU-instance snapshot.
- **`demo_gpu_farm`**: a Ray GPU-sharing farm (fractional or whole L4, local or on a remote g6). It showed that GPU sharing works, and that a non-MIG L4 wedges under sustained nvcc load.
- **`demo_cpu_branching`**: the real Claude Code CLI as harness, where one episode fans out into sub-agent and compaction leaves (mean fan-out ~5, `eval/swe` 0.47). Its advantages were 0 because of a grouping bug (the group was keyed by `sample.index` rather than `index // n_samples_per_prompt`). The bug was fixed, but the demo was never re-run.
- **`demo_cpu_swebench`** (SWE-bench Verified on the same pool code): one Runtime per instance on its Epoch AI arm64 image, reward from the hidden FAIL_TO_PASS + PASS_TO_PASS tests. It had **20 arm64 instances** and scored **reward 0** with Qwen3-4B. It was removed on 2026-09-30, and the planned 404-instance expansion was dropped. The 2 GB cap was probed on the way: a 2041.7 MiB push was accepted and a 2061.7 MiB push was rejected. Its EKS/ECS twins, the same task on self-managed containers, went with it.
- **Convergence scripts 01-19** and the shared top-level code: the library `code/agentcore_slime_demo` plus `code/deploy`, `code/scripts` and `code/tests`. The surviving demo vendors its own copy of the library and deploy helpers.

9 Open problems in production agentic RL — infrastructure and algorithms

The workflow in this report is deliberately small — one policy, a harness that runs inside a per-episode sandbox. Running agentic RL at *frontier* scale surfaces a set of recurring hard problems that the open reports (2024–2026) now describe in detail. This section catalogs them in two groups — infrastructure/systems and algorithms — with the mitigations people actually ship, and notes where this project's design already meets one and where it remains open.

9.1 Infrastructure and systems

Long-tail stragglers and partial rollouts. Rollout lengths are heavily right-skewed, so a synchronous step idles the whole cluster waiting on the last few long trajectories — and generation is more than 90 % of RL runtime [20]. Mitigations: *partial rollouts* that cap output and resume the unfinished portion next iteration (Kimi k1.5 [17]; Kimi K2 [5]); *active partial rollouts* that over-provision and recycle interrupted requests (APRIL, +22–44 % rollout throughput, now in slime [28]); *interruptible generation* that resumes decoding after a weight update (AReaL [18]); *skewness-aware routing* of long prompts (StreamRL [20]); and *dynamic sampling* that drops zero-gradient groups (DAPO [14]).

Weight-sync cost and staleness. Trainer and inference are separate backends; after each optimizer step, new weights must reach every rollout replica. Synchronously this is a serialization bubble; at 1T scale it moves hundreds of GB. Mitigations: *fast weight transfer* (Moonshot's checkpoint-engine updates all of Kimi-K2's ~1T parameters in ~20 s [29]); *zero-redundancy resharding* between train and generate layouts (veRL / HybridFlow [19]); and *decoupled, "one-step-off" async RL with a bounded staleness* and a staleness-aware objective (AReaL, up to ~2.77× [18]; Asynchronous RLHF [21]). *This project uses slime's fully-async rollout in the Terminal-Bench demo, so staleness is the accepted cost — and it is exactly the signal that is instrumented-yet-unrecorded here (§"The Terminal-Bench demo in depth").*

Goodput vs MFU, and placement. An iteration interleaves a bandwidth-bound, long-tailed generation phase with a compute-bound training phase, so one engine's GPUs idle while the other works — which is why MFU is misleading for RL and practitioners reason in *goodput* (useful samples per GPU-second). *This project's measured MFU of 2–4 % (the removed MBPP run, §"History: removed demos (2026-09-30)") is a concrete instance of that gap.* Mitigations: *colocate-with-offload* (veRL / HybridFlow, the frontier default [19], used by Kimi K2 [5]) vs *disaggregated placement* for heterogeneous or elastic hardware (StreamRL, up to 2.66× [20]; slime supports both). *Here the 4-train + 4-inference colocation is the simple default; disaggregation is the lever once the sandbox or a second cluster enters the picture.*

Scaling execution sandboxes to tens of thousands. Every step runs untrusted code in an isolated sandbox, so the sandbox layer — not the GPU — often becomes the bottleneck: constant create/destroy, strong isolation, real startup cost. Kimi K2 runs more than 10,000 concurrent Kubernetes sandboxes for its coding / SWE RL [5].

Mitigations: *microVM isolation* (Firecracker-class, sub-second boot) and *managed remote code-exec backends* (E2B, Modal). *This is precisely AgentCore Runtime's per-session microVM — the seam this whole report builds on; the project scales it as a pool of 218 per-task Runtime ARNs with a fresh session per episode (§"The Terminal-Bench demo").*

Cache-aware routing. Rollout throughput lives on prefix KV-cache reuse; a round-robin balancer scatters prefix-sharing requests and collapses the hit rate — acute in agentic RL, where a group of trajectories shares one task prefix and multi-turn rollouts grow a long shared prefix. Mitigations: RadixAttention plus SGLang's cache-aware router (~1.9× throughput; hit rate ~20 %→~75 %) [27], and session-affinity routing for multi-turn (slime `consistent_hashing` [9]). *This is the reasoning behind §"Scaling the loop": one cache-aware SGLang router for inference, round-robin only on the sandbox side.*

Fault tolerance. One step couples training + inference + reward + thousands of live sandboxes, so the blast radius is large. Mitigations: *async checkpointing*; *worker heartbeats* (slime's `/health_generate` with restart-and-re-sync, `--use-fault-tolerance` [9]); and *sandbox recycling* with timeouts and rate limits so one bad tool call does not abort a rollout. *The project's heartbeat (§"Hiding the container launch: pre-open and heartbeat") addresses the main sandbox-side failure mode, idle reclamation; a GPU-instance snapshot probe for instance recycle was tried and removed.*

9.2 Algorithms

The train/inference logprob mismatch and off-policy correction. Even with identical parameters, the inference engine and the trainer assign different token probabilities (differing parallelism, precision, kernels), so "on-policy" RL is silently off-policy — biasing gradients and risking entropy collapse, and worse for long responses. Mitigations: *Truncated Importance Sampling* (clip π_learner/π_sampler, now in veRL) [22]; *fixing the precision root cause* by reverting to FP16 [23]; and *decoupled / staleness-aware PPO* [18]. *This project's TITO property (the proxy records the exact sampled `token_ids` + logprobs, no re-tokenization — §"Token-in-token-out") removes the tokenization half of the mismatch, but not the engine-vs-trainer numeric half; TIS is the missing piece, and the importance-sampling ratio is instrumented-but-unrecorded here.*

GRPO and its variants. GRPO drops PPO's critic by normalizing advantages within a group of K sampled responses [13], but has documented failure modes (entropy collapse, length bias, zero-gradient groups, MoE instability). Upgrades: *DAPO* (clip-higher ε_high≈0.28, dynamic sampling, token-level loss, overlong reward shaping) [14]; *Dr. GRPO* (removes the length and std normalization that inflate wrong-answer length) [16]; *GSPO* (a sequence-level, length-normalized importance ratio that stabilizes MoE RL and underpins Qwen3) [15]. *This project trains with GRPO; these are the drop-in improvements as runs scale up.*

Credit assignment over long horizons. An agentic episode is many steps but the verifiable reward is sparse and terminal, so trajectory-level GRPO mis-credits the actions inside a failed or successful trajectory. Mitigations: *hierarchical / turn-level grouping* (GiGPO, +9–12 % over GRPO on ALFWorld / WebShop) [26]; entropy-aware step attribution; and denser process rewards derived from outcome labels alone. *The Terminal-Bench demo uses a binary end-of-episode reward with `EPISODE_MAX_TURNS=8`; long-horizon families like Terminal-Bench and SWE are where turn-level credit, or partial credit from the fraction of tests passed, becomes necessary.*

Reward hacking and verifier trust. Any learned reward can be gamed, so frontier labs prefer rule-based verifiable rewards — DeepSeek-R1 explicitly avoids neural reward models to prevent reward hacking [2]. But verifiers are not automatically trustworthy: "Spurious Rewards" shows RLVR with *random* rewards still improved a Qwen math model, because a clipping bias amplifies pretraining behaviors rather than reward correctness — and the effect is model-dependent [24]. *The project's rewards are binary RLVR checkers run inside the sandbox (no LLM judge), which sidesteps neural-RM hacking; the caution is that a benchmark gain does not by itself prove the checker is doing the work.*

Entropy collapse. During long RL, entropy drops early and exploration dies; performance is effectively "traded from policy entropy" [25]. Mitigations: clip-higher [14]; covariance-targeted regularization (Clip-Cov / KL-Cov) that intervenes only on the few high-covariance tokens driving entropy decay [25]; and KL control or removal.

Curriculum, dynamic sampling, and the discard ratio. All-correct or all-wrong groups give zero advantage → wasted compute. Mitigations: *dynamic sampling* to keep only groups with 0 < correct < G (DAPO [14]) and *pass-rate difficulty curricula* (AdaRFT, up to 2× less training time [30]). *This is exactly where the project's `discard_ratio` metric would come alive — it is defined (goodput depends on it) but zero in the current runs.*

9.3 Takeaways

Most of these are already partly handled by the stack this report uses: slime provides async rollout, fault-tolerant SGLang workers, and cache-aware routing; AgentCore provides the isolated, scalable sandbox. The open work for a production run concentrates in three places — (1) turning on and *recording* the off-policy signals (staleness, importance-sampling ratio, discard ratio) and adding TIS; (2) moving from GRPO to a variant (DAPO / GSPO) as scale and response length grow; and (3) turn-level credit assignment for the long-horizon families. These are algorithm choices layered on top of the infrastructure the demo already exercises.

10 Summary

- **AgentCore Runtime is a workable RL sandbox for per-task environments.** A Runtime is a deployed image, a session is one microVM, and a fresh session per episode gives each episode its own kernel with no state carried over. The arm64 and 2 GB image limits are hard, and they decide which tasks fit.
- **slime needs no fork.** One generate hook, a sandbox client (one `InvokeAgentRuntime` per operation), and the mini_claude loop running inside the session, dialling the trainer's adapter back over the VPC, are the whole integration. Token-in-token-out keeps the trained tokens exactly the sampled ones.
- **Terminal-Bench is the demo.** 218 Runtimes are built from the tasks' own Dockerfiles, 166 of them oracle-validated. Rows are routed strictly to their Runtime. Look-ahead pre-open takes the cold start off the critical path (open wait p95 0 at the cost of ~95 % idle pre-opened time), and the heartbeat keeps sessions alive across the weight-sync stall. The harness tier scales the agent loop with the sandbox.
- **What is open.** Reward at 4B is sparse (4/382 solved on 2026-09-23; 23/449 in the clean 2026-09-30 re-run, eval 1/62). Convergence run 20, plus a curriculum or partial-credit reward, is the next step.

11 References — papers & models

Agentic RL & models. [1] Agent Lightning — Harnessed Agentic RL. https://arxiv.org/abs/2508.03680 ·[2] DeepSeek-R1. https://arxiv.org/abs/2501.12948 ·[3] Qwen3 Technical Report. https://arxiv.org/abs/2505.09388 ·[4] Llama-Nemotron. https://arxiv.org/abs/2505.00949 ·[5] Kimi K2. https://arxiv.org/abs/2507.20534 ·[6] SWE-bench. https://arxiv.org/abs/2310.06770 ·[6b] SWE-bench README (arm64 “experimental”; `--task-repo` local Buildx build). https://github.com/SWE-bench/SWE-bench ·[6c] Terminal-Bench. https://github.com/laude-institute/terminal-bench ·[7] KernelBench. https://huggingface.co/datasets/ScalingIntelligence/KernelBench ·[8] MBPP. https://arxiv.org/abs/2108.07732 ·[9] slime. https://github.com/THUDM/slime ·[10] AceReason-Nemotron: Advancing Math and Code Reasoning through RL. https://arxiv.org/abs/2505.16400 ·[11] LiveCodeBench. https://arxiv.org/abs/2403.07974 ·[12] NVIDIA Nemotron Post-Training Dataset v2. https://huggingface.co/datasets/nvidia/Nemotron-Post-Training-Dataset-v2

12 References — AgentCore docs (1/2)

Official AgentCore docs. [D1] What is AgentCore. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html ·[D2] Host with Runtime (isolation, compute types). https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/agents-tools-runtime.html ·[D3] Runtime how it works (idle 15 m / max 8 h). https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-how-it-works.html ·[D4] Lifecycle settings. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-lifecycle-settings.html ·[D5] Instances (GPU). https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-instances-how-it-works.html ·[D6] Code Interpreter. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/code-interpreter-tool.html ·[D7] Code Interpreter session. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/code-interpreter-session-characteristics.html

13 References — AgentCore docs (2/2)

[D8] InvokeAgentRuntime. https://docs.aws.amazon.com/bedrock-agentcore/latest/APIReference/API_InvokeAgentRuntime.html ·[D9] CreateAgentRuntime. https://docs.aws.amazon.com/bedrock-agentcore-control/latest/APIReference/API_CreateAgentRuntime.html ·[D10] NetworkConfiguration. https://docs.aws.amazon.com/bedrock-agentcore-control/latest/APIReference/API_NetworkConfiguration.html ·[D11] VPC interface endpoints. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/vpc-interface-endpoints.html ·[D12] StopRuntimeSession. https://docs.aws.amazon.com/bedrock-agentcore/latest/APIReference/API_StopRuntimeSession.html ·[D13] Observability. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/observability.html ·[D17] Runtime service contract (port 8080, /invocations, /ping). https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-service-contract.html

14 References — AgentCore SDK, CLI & samples

AWS code, cross-checked against the docs. [D14] AgentCore samples (`awslabs/agentcore-samples`). https://github.com/awslabs/agentcore-samples ·[D15] AgentCore Python SDK (`bedrock-agentcore`; `BedrockAgentCoreApp` / `@app.entrypoint`). https://github.com/aws/bedrock-agentcore-sdk-python ·[D16] AgentCore CLI

(`@aws/agentcore`, recommended) https://www.npmjs.com/package/@aws/agentcore ·legacy starter toolkit https://github.com/aws/bedrock-agentcore-starter-toolkit

15 References — production agentic RL (open problems)

RL algorithms & systems. [13] DeepSeekMath (GRPO). https://arxiv.org/abs/2402.03300 ·[14] DAPO. https://arxiv.org/abs/2503.14476 ·[15] GSPO (Group Sequence Policy Optimization). https://arxiv.org/abs/2507.18071 ·[16] Understanding R1-Zero-Like Training (Dr. GRPO). https://arxiv.org/abs/2503.20783 ·[17] Kimi k1.5. https://arxiv.org/abs/2501.12599 ·[18] AReaL (asynchronous RL system). https://arxiv.org/abs/2505.24298 ·[19] HybridFlow (veRL). https://arxiv.org/abs/2409.19256 ·[20] StreamRL (disaggregated stream generation). https://arxiv.org/abs/2504.15930 ·[21] Asynchronous RLHF. https://arxiv.org/abs/2410.18252 ·[22] Truncated Importance Sampling / train–inference mismatch (Yao et al.). https://fengyao.notion.site/off-policy-rl ·[23] Defeating the Training–Inference Mismatch via FP16. https://arxiv.org/abs/2510.26788 ·[24] Spurious Rewards. https://arxiv.org/abs/2506.10947 ·[25] The Entropy Mechanism of RL. https://arxiv.org/abs/2505.22617 ·[26] GiGPO (hierarchical group advantages). https://arxiv.org/abs/2505.10978 ·[27] SGLang / RadixAttention. https://arxiv.org/abs/2312.07104 — cache-aware router: https://lmsys.org/blog/2024-12-04-sglang-v0-4/ ·[28] APRIL (active partial rollouts). https://arxiv.org/abs/2509.18521 ·[29] Moonshot checkpoint-engine. https://github.com/MoonshotAI/checkpoint-engine ·[30] AdaRFT (difficulty curriculum). https://arxiv.org/abs/2504.05520 ·[31] DSec: DeepSeek Elastic Compute, a sandbox platform for agentic RL. https://arxiv.org/abs/2609.22978